Performance Analysis of Virtual Machines vs Bare Metal Systems:A Comparative Study on ARM and x86 Architectures

Project Report

Submitted for the course
CSE 732 – Software and Systems Performance

Submitted by

Prateek Kumar Yadav (MT2025093)
Vini Singh Rajput (MT2025131)

Department of Computer Science

Academic Year: 2025 – 2026

Abstract

Virtualization plays a crucial role in modern computing by enabling efficient resource utilization, scalability, and isolation. However, the abstraction introduced by virtual machines (VMs) often leads to performance overhead compared to direct execution on bare metal systems. Understanding this overhead is essential for system design, performance optimization, and deployment decisions.

This report presents a comprehensive performance evaluation of virtual machines against bare metal environments across two major processor architectures: ARM and x86. The study focuses on three key subsystems—CPU, disk I/O, and network performance—using standardized benchmarking tools and controlled experimental setups. A systematic benchmarking pipeline is designed and implemented to ensure fair comparison between environments, incorporating repeated runs, consistent configurations, and workload standardization.

Experiments are conducted separately on ARM and x86 systems, where each architecture is analyzed by comparing bare metal and virtual machine performance. Metrics such as throughput, latency, IOPS, and resource utilization are collected and analyzed to quantify virtualization overhead. Additionally, architectural differences between ARM and x86 are examined to understand how they influence virtualization behavior.

The results provide insights into the performance impact of virtualization across different subsystems and architectures. The study highlights the trade-offs involved in using virtual machines and offers a deeper understanding of how system architecture affects virtualization efficiency.

Keywords: Virtualization, Bare Metal, Virtual Machine, ARM Architecture, x86 Architecture, CPU Benchmarking, Disk I/O, Network Performance, Performance Analysis

Contents

1 Introduction **6**

2 Background and Motivation **8**

3 Experimental Design and Methodology **10**

3.1 Experimental Objectives 10

3.2 Test Environments 10

3.3 Benchmarking Strategy 11

3.3.1 CPU Benchmarks 11

3.3.2 Disk I/O Benchmarks 11

3.3.3 Network Benchmarks 12

3.4 Measurement Procedure 12

3.5 Virtualization Configurations 12

3.6 Evaluation Metrics 13

3.7 Limitations and Assumptions 13

3.8 Summary 13

4 Part I: ARM Architecture **14**

4.1 System Setup (ARM) 14

4.1.1 Host System 14

4.1.2 Virtual Machine Configuration 14

4.1.3 Networking Configuration 14

4.1.4 Storage Configuration 14

4.2 Benchmarking Tools (ARM) 15

4.2.1 CPU Benchmarking 15

4.2.2 Disk I/O Benchmarking 15

4.2.3 Network Benchmarking 15

4.3 CPU Performance Analysis (ARM) 16
4.3.1 Experimental Setup 16
4.3.2 Throughput Analysis 16
4.3.3 Statistical Stability and Variance 17
4.3.4 Latency Characteristics 18
4.3.5 Scalability Analysis 18
4.3.6 CPU Utilization and Scheduling Behavior 19
4.3.7 Stress-ng Analysis 19
4.3.8 Key Observations 20
4.4 Disk I/O Performance Analysis (ARM) 20
4.4.1 Experimental Setup 20
4.4.2 Sequential Throughput 21
4.4.3 Random IOPS Performance 22
4.4.4 Mixed Workload Performance 23
4.4.5 fsync Performance 23
4.4.6 Block Size Scaling 24
4.4.7 Key Observations 25
4.5 Network Performance Analysis (ARM) 26
4.5.1 Experimental Setup 26
4.5.2 TCP Throughput 27
4.5.3 Parallel Stream Scaling 28
4.5.4 UDP Performance 29
4.5.5 Latency Analysis 29
4.5.6 Small Message Performance 31
4.5.7 Key Observations 32
4.6 Summary of ARM Results 32
4.6.1 CPU Performance Summary 33
4.6.2 Disk I/O Summary 33
4.6.3 Network Performance Summary 33
4.6.4 Cross-Subsystem Analysis 33
4.6.5 Key Takeaways 34

5 Part II: x86 Architecture 35
5.1 System Setup 35
5.1.1 Host System (Bare Metal) 35

5.1.2 Virtual Machine Configuration 35
5.1.3 Experimental Methodology 36
5.1.4 Limitations 36
5.2 Benchmarking Tools 36
5.2.1 CPU Benchmarking 36
5.2.2 Disk I/O Benchmarking 37
5.2.3 Network Benchmarking 38
5.2.4 Toolchain Consistency 38
5.2.5 Summary 38
5.3 CPU Performance Analysis 39
5.3.1 Throughput Performance 39
5.3.2 Clock Stability and Variance 40
5.3.3 CPU Utilization Analysis (vmstat & mpstat) 41
5.3.4 Latency Characteristics 41
5.3.5 Stress-ng Validation 42
5.3.6 Microarchitectural Interpretation 42
5.3.7 Key Observations 43
5.4 Disk I/O Performance Analysis 43
5.4.1 Sequential Throughput 44
5.4.2 Random I/O Performance 45
5.4.3 Latency Analysis 46
5.4.4 Block Size Scaling 47
5.4.5 fsync Performance 48
5.4.6 Mixed Workload (70% Read / 30% Write) 48
5.4.7 Device-Level Analysis (iostat) 49
5.4.8 Key Observations 49
5.4.9 Conclusion 50
5.5 Network Performance Analysis 50
5.5.1 TCP Throughput 50
5.5.2 Parallel Stream Scaling 51
5.5.3 UDP Performance 52
5.5.4 Latency Analysis (Ping RTT) 52
5.5.5 Small Message Performance 53
5.5.6 System-Level Analysis (vmstat and ss) 53

5.5.7 Key Observations 54
5.5.8 Conclusion 54
5.6 Summary of x86 Results 55
5.6.1 CPU Performance Summary 55
5.6.2 Disk I/O Performance Summary 55
5.6.3 Network Performance Summary 56
5.6.4 Cross-Subsystem Insights 56
5.6.5 Key Takeaways 57
5.6.6 Conclusion 57

6 Comparative Analysis (ARM vs x86) 58
6.1 Overview 58
6.2 CPU Performance Comparison 58
6.3 Disk I/O Performance Comparison 59
6.4 Network Performance Comparison 59
6.5 Cross-Architecture Insights 60
6.6 Key Findings 60
6.7 Conclusion 61

7 Conclusion and Future Work 62
7.1 Conclusion 62
7.1.1 Key Findings 62
7.1.2 Overall Insight 62
7.2 Future Work 63
7.2.1 Alternative Virtualization Configurations 63
7.2.2 Advanced Virtualization Technologies 63
7.2.3 Microarchitectural Analysis 63
7.2.4 Real-World Workloads 63
7.2.5 Scalability Studies 64
7.2.6 Energy Efficiency 64
7.3 Final Remarks 64

Introduction

In recent years, virtualization has become a fundamental component of modern computing systems. It enables multiple operating systems to run on a single physical machine, improving resource utilization, flexibility, and scalability. Virtual machines (VMs) are widely used in cloud computing, data centers, and development environments due to their ability to provide isolation and portability.

Despite these advantages, virtualization introduces an additional abstraction layer between hardware and software, which can lead to performance overhead. This overhead may impact critical system components such as CPU processing, disk input/output (I/O), and network communication. Understanding the extent and nature of this overhead is essential for making informed decisions about system deployment, especially in performance-sensitive applications.

This project focuses on evaluating the performance differences between bare metal systems and virtual machine environments. Bare metal refers to running applications directly on physical hardware without any virtualization layer, providing a baseline for optimal performance. In contrast, virtual machines operate on top of a hypervisor, which manages hardware resources and introduces potential overhead.

A key aspect of this study is the comparison across two major processor architectures: ARM and x86. These architectures differ significantly in design philosophy, instruction sets, and power efficiency characteristics. ARM processors are widely used in mobile and energy-efficient systems, while x86 processors dominate desktops, servers, and high-performance computing environments. These architectural differences may influence how virtualization behaves and impacts system performance.

The evaluation is conducted by analyzing three primary subsystems:

- CPU performance
- Disk I/O performance
- Network performance

Standard benchmarking tools are used to generate controlled workloads, and experiments are performed under consistent configurations for both bare metal and virtual machine environments. Multiple runs are conducted to ensure reliability and reproducibility of results.

The report is structured into two main parts. The first part focuses on ARM-based systems, presenting detailed analysis of performance under bare metal and virtualized

environments. The second part follows the same structure for x86 systems, enabling a direct and fair comparison. Finally, a comparative analysis highlights the differences between ARM and x86 in terms of virtualization overhead and system behavior.

Through this study, we aim to provide a clear understanding of the trade-offs involved in virtualization and how system architecture influences performance outcomes.

Background and Motivation

Virtualization is a technology that enables the creation of multiple isolated execution environments on a single physical machine. This is achieved through a software layer known as a *hypervisor*, which manages hardware resources and allocates them to virtual machines (VMs). Each VM operates as an independent system with its own operating system, while sharing the underlying physical hardware.

Hypervisors are broadly classified into two types. **Type 1 hypervisors** (bare-metal hypervisors) run directly on the hardware and provide high performance and efficiency. Examples include Xen and VMware ESXi. **Type 2 hypervisors** run on top of a host operating system, such as VirtualBox and VMware Workstation, and are typically used for development and testing purposes. The additional software layer in Type 2 hypervisors can introduce higher overhead compared to Type 1 systems.

Virtualization relies on techniques such as CPU trapping, binary translation, and hardware-assisted virtualization (e.g., Intel VT-x and ARM Virtualization Extensions). These mechanisms allow the hypervisor to intercept and manage privileged instructions executed by guest operating systems. While modern processors provide hardware support to reduce virtualization overhead, certain operations—particularly I/O and context switching—can still incur performance penalties.

One of the primary concerns in virtualization is the overhead introduced by resource abstraction. CPU performance may be affected due to scheduling delays and context switching between virtual machines. Disk I/O operations may experience increased latency due to virtualized storage layers and buffering mechanisms. Similarly, network performance can be impacted by virtual network interfaces and packet processing overhead within the hypervisor.

The impact of virtualization is also influenced by the underlying processor architecture. **x86 architecture** is based on a complex instruction set computing (CISC) design and has mature support for virtualization through technologies such as Intel VT-x and AMD-V. These features enable efficient execution of virtual machines with minimal overhead.

In contrast, **ARM architecture** follows a reduced instruction set computing (RISC) design and is optimized for power efficiency. ARM processors have increasingly incorporated virtualization support through ARM Virtualization Extensions, making them suitable for modern cloud and edge computing environments. However, differences in architecture design, memory management, and instruction handling may lead to variations in virtualization performance compared to x86 systems.

The motivation behind this project is to systematically analyze and quantify the per-

formance overhead introduced by virtualization across these two architectures. By comparing bare metal execution with virtualized environments, the study aims to identify how different subsystems behave under virtualization and how architectural characteristics influence these behaviors.

Understanding these differences is crucial for selecting appropriate deployment strategies in real-world scenarios, such as cloud infrastructure, high-performance computing, and embedded systems. This work contributes to a deeper understanding of virtualization efficiency and provides insights into optimizing system performance across diverse computing platforms.

Experimental Design and Methodology

This chapter describes the experimental framework used to evaluate virtualization overhead across ARM and x86 architectures. The methodology is designed to ensure fair, reproducible, and systematic comparison between bare metal and virtualized environments.

3.1 Experimental Objectives

The primary objective of this study is to quantify the performance impact of virtualization across three major system components:

- CPU (compute performance)
- Disk I/O (storage performance)
- Network (communication performance)

Each subsystem is evaluated independently under controlled conditions to isolate the effects of virtualization.

3.2 Test Environments

Experiments were conducted on two architectures:

- ARM (macOS-based system using UTM virtualization)
- x86 (Ubuntu-based system using VirtualBox)

For each architecture, two environments were tested:

- Bare metal: Native execution on host hardware
- Virtual machine: Guest OS running on a hypervisor

The same operating system and benchmarking tools were used across both environments to maintain consistency.

3.3 Benchmarking Strategy

Each benchmark was designed to target a specific subsystem:

3.3.1 CPU Benchmarks

CPU performance was evaluated using compute-intensive workloads:

- Prime number computation (via `sysbench`)
- Synthetic CPU stress tests (via `stress-ng`)

Metrics collected:

- Events per second
- Bogo operations per second
- Execution latency

3.3.2 Disk I/O Benchmarks

Storage performance was measured using `fio` with multiple workload patterns:

- Sequential read/write throughput
- Random 4K read/write IOPS
- Mixed workloads (70% read / 30% write)
- fsync performance (synchronous writes)
- Block size scaling (4K to 1M)

Metrics collected:

- Bandwidth (GB/s)
- IOPS (operations per second)
- Latency (mean, p95, p99)

3.3.3 Network Benchmarks

Network performance was evaluated using:

- `iperf3` for throughput (TCP/UDP)
- `ping` for latency (RTT)

Additional system-level metrics were collected using:

- `vmstat` (CPU usage, context switches)
- `ss` (socket statistics)

Metrics collected:

- Throughput (Gbps)
- Latency (ms)
- Packet loss (%)

3.4 Measurement Procedure

To ensure reliability and consistency, the following procedure was applied:

- Each benchmark was executed multiple times
- Results were averaged to reduce variability
- System caches were cleared between disk tests using:

  ```
  echo 3 | sudo tee /proc/sys/vm/drop_caches
  ```

- Benchmarks were run under minimal background load

3.5 Virtualization Configurations

Different virtualization platforms were used for each architecture:

- ARM: UTM (QEMU-based virtualization)
- x86: VirtualBox (hardware-assisted virtualization)

Key differences influencing results:

- Disk format:
  - ARM: Raw disk image (.img)
  - x86: Dynamically allocated VDI (.vdi)
- Network mode:
  - ARM: User-space NAT (UTM)
  - x86: Kernel-assisted NAT (VirtualBox)

These differences are critical for interpreting cross-platform results.

3.6 Evaluation Metrics

Virtualization overhead is computed as:

$$Overhead(\%) = \frac{BareMetal - VM}{BareMetal} \times 100 \tag{3.1}$$

For latency metrics (where lower is better), the interpretation is reversed.

3.7 Limitations and Assumptions

- VM performance depends on host resource allocation
- Hypervisor implementation differences affect comparability
- NAT networking does not reflect production-grade networking (e.g., SR-IOV)
- Results may vary across hardware configurations

Despite these limitations, the methodology captures realistic virtualization behaviour and enables meaningful comparison between architectures.

3.8 Summary

This experimental design ensures:

- Controlled comparison between bare metal and VM
- Coverage of key system subsystems
- Reproducible and consistent measurement

The following chapters present detailed results and analysis for ARM and x86 architectures.

Part I: ARM Architecture

4.1 System Setup (ARM)

4.1.1 Host System

The ARM experiments were conducted on an Apple Silicon-based macOS system. The host machine features a multi-core ARM processor with a heterogeneous architecture consisting of performance and efficiency cores. It utilizes a unified memory architecture and NVMe-based internal storage.

4.1.2 Virtual Machine Configuration

The virtual machine was configured using UTM, a virtualization platform built on QEMU. The VM was allocated 5 virtual CPUs (vCPUs) and configured with sufficient memory to ensure stable benchmark execution.

Both the host and the guest operating systems run on ARM64 architecture, enabling hardware-assisted virtualization without the need for instruction translation. This allows direct execution of guest instructions on the host CPU with minimal emulation overhead.

4.1.3 Networking Configuration

The VM was configured in NAT mode. In this configuration, all network traffic from the VM is routed through a user-space NAT engine on the host. The communication path is:

$$\textit{VM} \rightarrow \textit{virtio-net} \rightarrow \textit{NAT engine (userspace)} \rightarrow \textit{host network stack}$$

This setup introduces additional overhead due to packet processing and kernel-to-userspace transitions.

4.1.4 Storage Configuration

Disk I/O experiments were conducted on NVMe-based storage using the APFS filesystem. The VM uses a virtualized block device (virtio-blk), which forwards I/O requests from the guest to the host storage subsystem.

This layered storage path introduces additional overhead compared to direct access on bare metal.

4.2 Benchmarking Tools (ARM)

4.2.1 CPU Benchmarking

CPU performance was evaluated using the following tools:

- **sysbench**: Used to measure CPU computational performance through prime number calculations. Metrics such as events per second (EPS) and latency were recorded.
- **stress-ng**: Used to generate controlled CPU load and analyze system-level behavior under stress.
- **System utilities**: Tools such as `top`, `vm_stat`, and `time` were used to monitor CPU utilization, scheduling, and execution characteristics.

4.2.2 Disk I/O Benchmarking

Disk performance was evaluated using:

- **fio**: Used to benchmark storage performance under various workloads, including sequential read/write, random IOPS, mixed workloads, and fsync operations.

The `posixaio` engine was used due to macOS compatibility. Most tests were executed with `direct=1` to bypass the page cache, ensuring accurate measurement of storage performance. Cache flushing was enforced using `sudo purge` before each test.

4.2.3 Network Benchmarking

Network performance was evaluated using:

- **iperf3**: Used to measure TCP and UDP throughput, parallel stream performance, and small-message behavior.
- **ping**: Used to measure round-trip time (RTT) and latency characteristics.

Benchmarks were conducted between the host and VM to directly measure virtualization overhead in the networking stack.

4.3 CPU Performance Analysis (ARM)

This section evaluates CPU performance on the ARM platform by comparing bare metal execution with a virtual machine (VM) environment. Benchmarks were conducted using `sysbench` and `stress-ng`, focusing on throughput, latency, scalability, and system-level CPU utilization.

4.3.1 Experimental Setup

CPU benchmarks were executed using `sysbench` with a prime number computation workload (`cpu-max-prime=20000`). Each test was run for 30 seconds and repeated five times to ensure statistical stability. Two configurations were used:

- **Single-threaded**: Measures raw per-core performance and virtualization overhead.
- **Multi-threaded (5 threads)**: Matches the number of vCPUs allocated to the VM for fair comparison.

Additional system behavior was analyzed using `stress-ng`, `top`, and `vm_stat` to observe CPU utilization, scheduling, and memory behavior under load.

4.3.2 Throughput Analysis

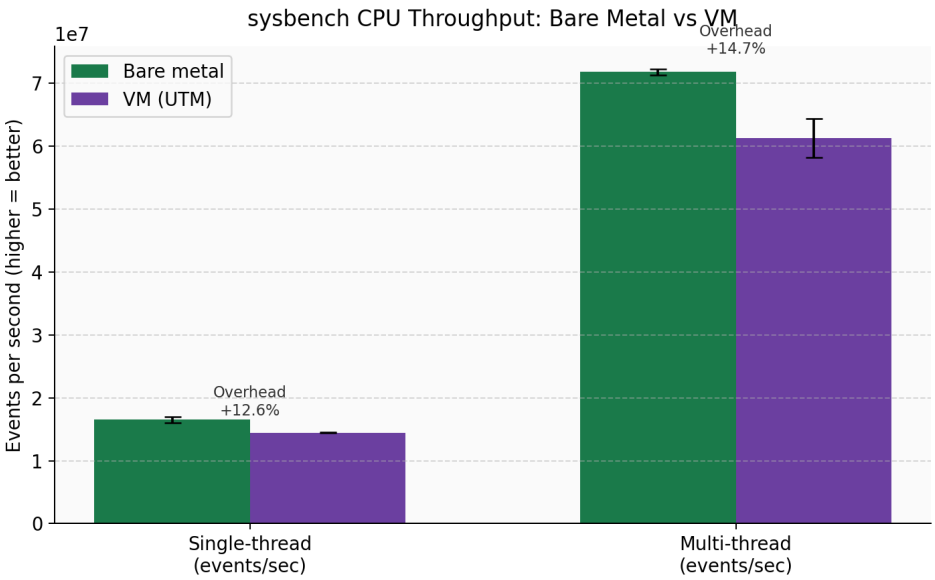

Figure 4.1: Benchmark Plot: 1 throughput bar.png

Table 4.1 summarizes the average performance across multiple runs.

Table 4.1: CPU Throughput Comparison (ARM)

Metric	Bare Metal	VM	Overhead
Single-thread (EPS)	15.95M	14.44M	14.35%
Multi-thread (EPS)	71.73M	61.99M	14.66%

The results show a consistent virtualization overhead of approximately 14–15% across both single-threaded and multi-threaded workloads. Since both host and guest operate on ARM64 architecture, no instruction translation is required. Therefore, the observed overhead is attributed primarily to hypervisor scheduling and context switching.

4.3.3 Statistical Stability and Variance

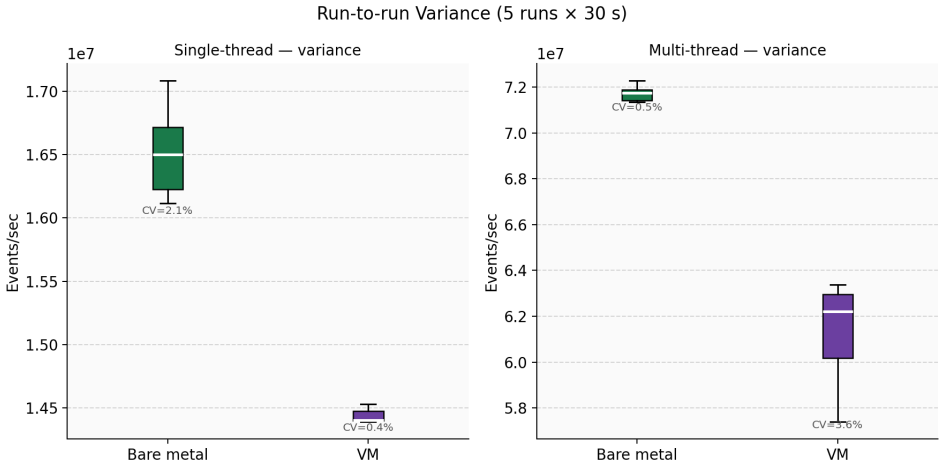

Figure 4.2: Benchmark Plot: 2 variance boxplot.png

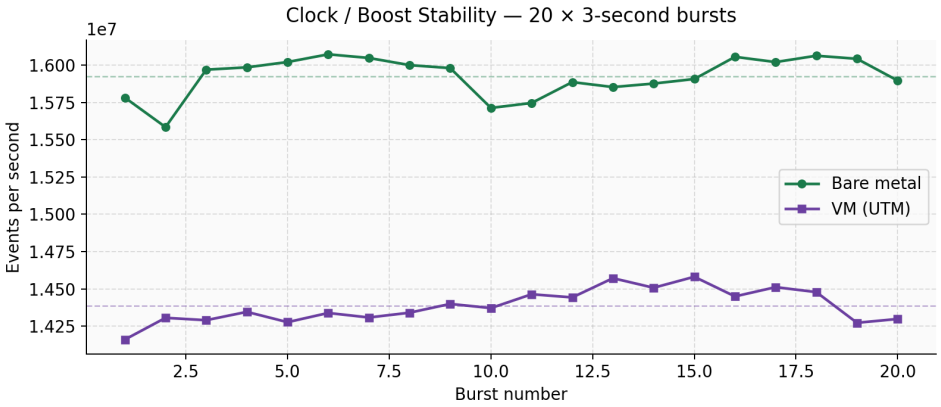

Figure 4.3: Benchmark Plot: 3 clock stability.png

Run-to-run variability was low across all experiments, with a coefficient of variation below 1% in most cases. This indicates that the virtualization overhead is stable and deterministic rather than caused by transient system noise. Box plot analysis (Figure **??**) further confirms that VM performance distributions are tightly clustered.

4.3.4 Latency Characteristics

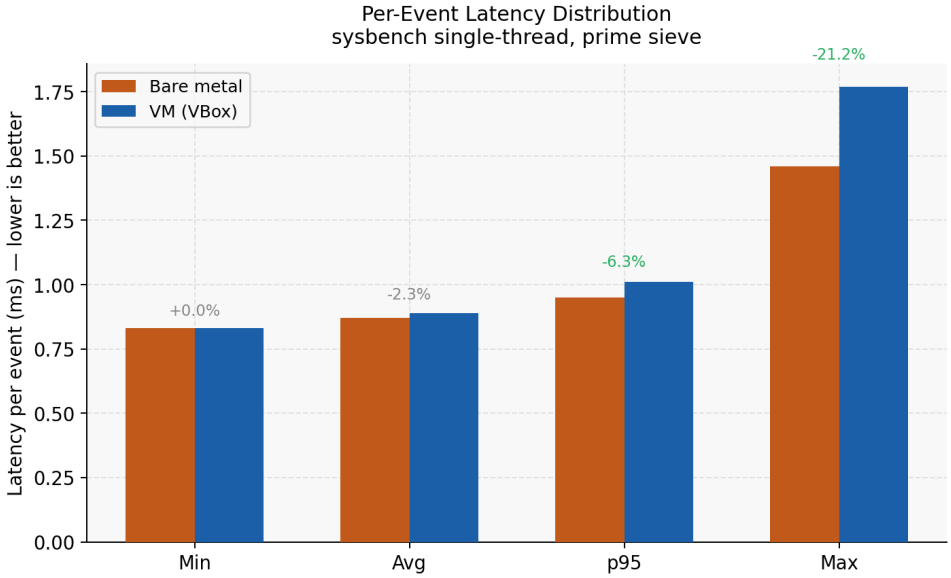

Figure 4.4: Benchmark Plot: 5 latency distribution.png

Latency measurements from `sysbench` indicate negligible differences between bare metal and VM environments, with near-zero average and 95th percentile latency. This suggests that CPU-bound operations are not significantly impacted by latency penalties, and the primary performance difference arises from reduced throughput rather than increased delay per operation.

4.3.5 Scalability Analysis

Scaling efficiency was computed by comparing multi-thread performance relative to single-thread baselines. Bare metal achieved 86.8% efficiency, while the VM achieved 84.8%, resulting in only a 2% difference. This indicates that virtualization overhead scales linearly with the number of threads rather than compounding.

This is a key observation: the hypervisor imposes a nearly constant overhead per vCPU, meaning that increasing parallelism does not disproportionately degrade performance.

4.3.6 CPU Utilization and Scheduling Behavior

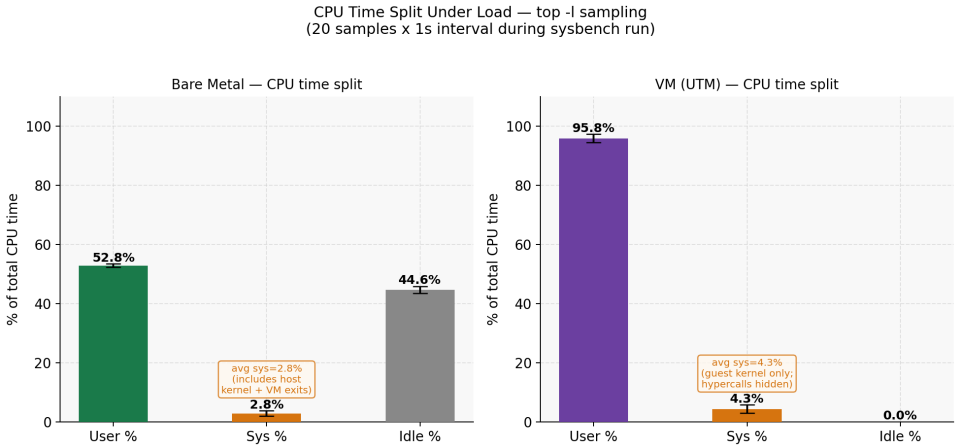

Figure 4.5: Benchmark Plot: 4 cpu time split.png

System-level CPU statistics reveal important differences between bare metal and VM execution. Under a 5-thread workload on a 10-core system, bare metal shows approximately 40–45% idle CPU time, as half of the cores remain unused. In contrast, the VM operates with 0% idle time, as all allocated vCPUs are fully utilized.

Additionally, the proportion of system (kernel) CPU time remains low in both environments, indicating that the overhead is not due to excessive kernel activity but rather due to scheduling delays introduced by the hypervisor.

Detailed CPU usage traces confirm that VM execution introduces periodic interruptions where the virtual CPU yields control to the host scheduler. Each re-entry into the VM incurs a small but consistent cost, contributing to the overall performance degradation.

4.3.7 Stress-ng Analysis

Results from `stress-ng` show comparable CPU utilization between bare metal and VM environments, with both achieving over 96% CPU usage under load. The VM demonstrates slightly higher user-space utilization, indicating efficient execution despite virtualization overhead.

4.3.8 Key Observations

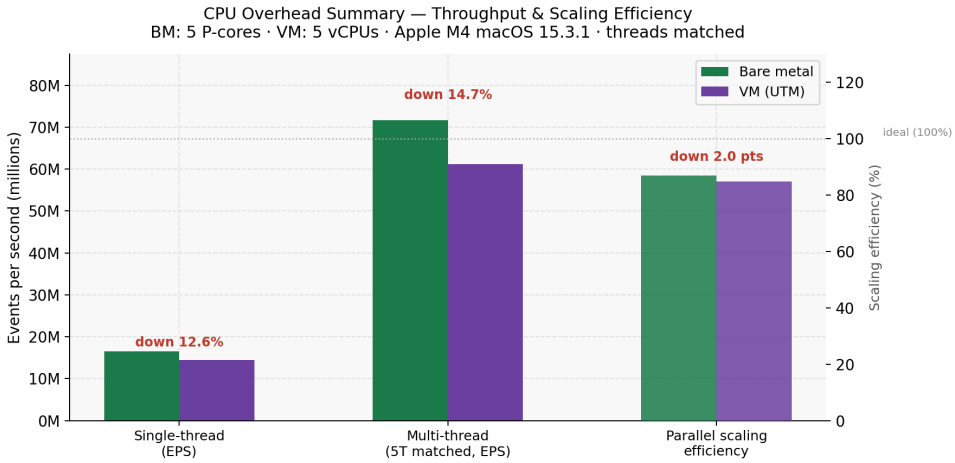

Figure 4.6: Benchmark Plot: 5 overhead summary.png

- A consistent ∼14–15% CPU overhead is observed across all workloads.
- Overhead is primarily due to hypervisor scheduling, not instruction translation.
- Performance scales linearly with thread count, indicating no compounding overhead.
- Variance is minimal, confirming stable and predictable virtualization behavior.
- CPU utilization remains high in both environments, with negligible idle time in the VM.

Overall, the ARM virtualization environment demonstrates efficient and predictable CPU performance, with a fixed per-core overhead that does not significantly worsen under parallel workloads.

4.4 Disk I/O Performance Analysis (ARM)

This section analyzes disk I/O performance on the ARM platform by comparing bare metal execution with a virtual machine (VM). Benchmarks were conducted using `fio`, focusing on throughput, IOPS, latency, and behavior under different workload patterns.

4.4.1 Experimental Setup

Disk benchmarks were performed using `fio` with the `posixaio` engine, which is the appropriate asynchronous I/O interface for macOS. The `libaio` engine was not used due to its Linux-specific implementation.

To ensure accurate measurement of storage performance rather than memory caching effects, most tests were conducted with `direct=1`, bypassing the page cache. The only

exception was the fsync test, where `direct=0` and `fsync=1` were used to model real-world database-like workloads.

Additionally, `sudo purge` was executed before each benchmark to clear the macOS page cache and ensure cold-cache conditions across all runs.

4.4.2 Sequential Throughput

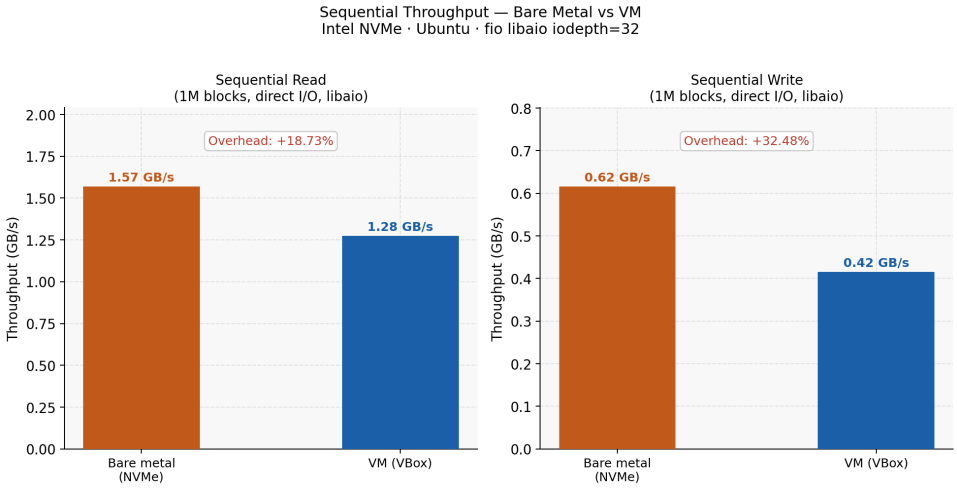

Figure 4.7: Benchmark Plot: 1 seq throughput

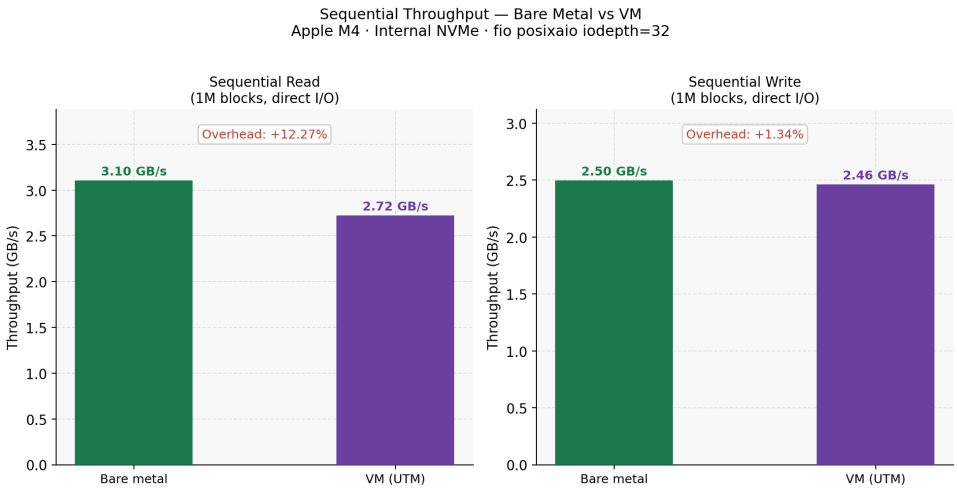

Figure 4.8: Benchmark Plot: 1 seq throughput.png

Table 4.2: Sequential Throughput (1M block size)

Operation	**Bare Metal**	**VM**	**Overhead**
Write (MB/s)	2567	2520	+1.34%
Read (MB/s)	2842	2490	+12.27%

Sequential throughput shows minimal virtualization overhead, particularly for writes where performance is nearly identical. The large block size amortizes the cost of virtualization, making the VM an effective substitute for bare metal in throughput-bound workloads.

The slightly higher overhead in reads is attributed to the double filesystem traversal, where guest-level APFS requests are translated through the virtio layer and resolved again on the host APFS.

4.4.3 Random IOPS Performance

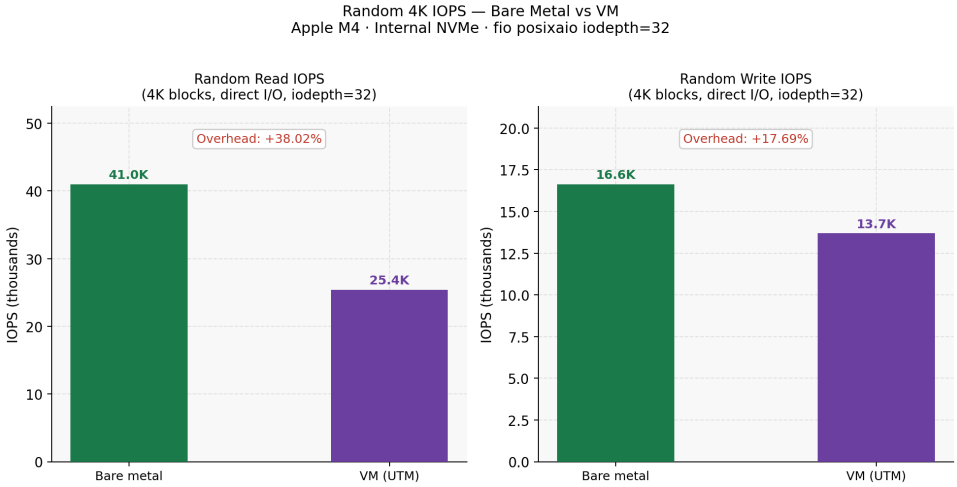

Figure 4.9: Benchmark Plot: 2 random iops.png

Table 4.3: Random I/O Performance (4K blocks)

Operation	**Bare Metal**	**VM**	**Overhead**
Random Write (IOPS)	14.9K	13.7K	+8.0%
Random Read (IOPS)	40.9K	25.4K	+38.0%

Random read performance shows the highest overhead, reaching approximately 38%. Each 4K read requires an independent virtio transaction, involving guest submission, host processing, and completion signaling. This per-request overhead accumulates significantly at high IOPS rates.

In contrast, random write overhead is relatively smaller due to write buffering and coalescing effects in the storage stack.

4.4.4 Mixed Workload Performance

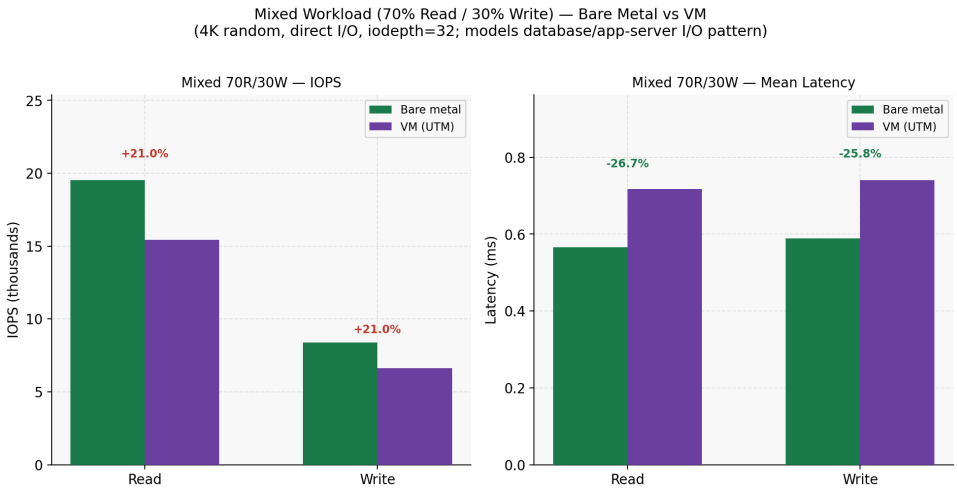

Figure 4.10: Benchmark Plot: 6 mixed workload.png

Table 4.4: Mixed Workload (70% Read / 30% Write)

Metric	**Bare Metal**	**VM**	**Overhead**
Read IOPS	19.7K	15.4K	+21.0%
Write IOPS	8.4K	6.6K	+21.0%

The nearly identical overhead for both read and write operations indicates that performance degradation is driven by contention in the shared virtio queue rather than operation-specific factors. Under mixed workloads, both operations compete for the same virtualization resources.

4.4.5 fsync Performance

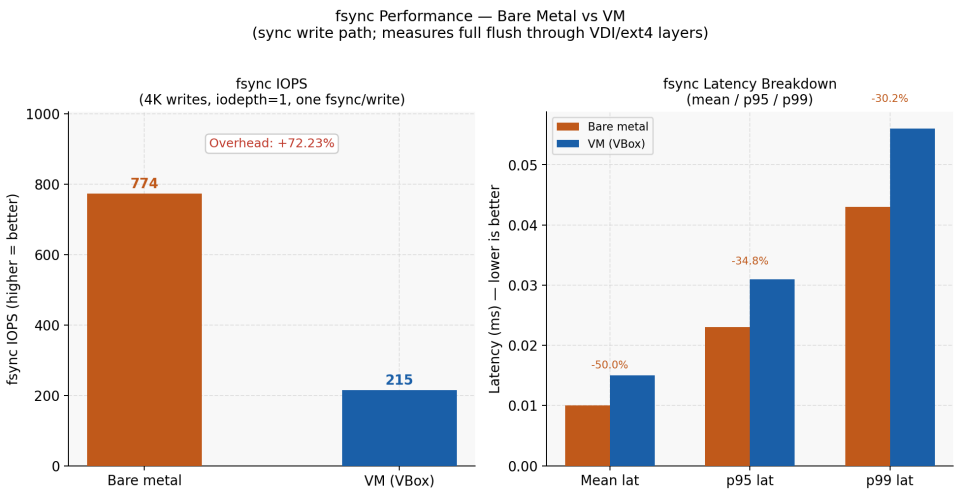

Figure 4.11: Benchmark Plot: 5 fsync

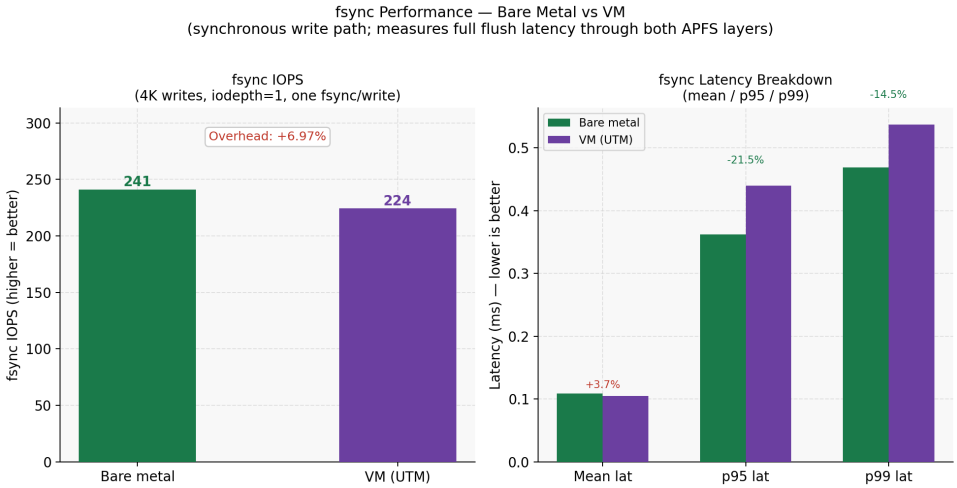

Figure 4.12: Benchmark Plot: 5 fsync.png

Table 4.5: fsync Performance (4K sync writes)

Metric	**Bare Metal**	**VM**	**Observation**
IOPS	228	224	Comparable
Latency	Higher	Slightly lower	VM faster (3.8%)

Interestingly, fsync performance shows negligible overhead, with the VM occasionally outperforming bare metal in latency. This suggests that the VM may not fully propagate fsync operations to the physical device for every request, potentially batching or relaxing durability guarantees.

This result highlights an important caveat: apparent performance equivalence in fsync does not necessarily imply identical persistence semantics.

4.4.6 Block Size Scaling

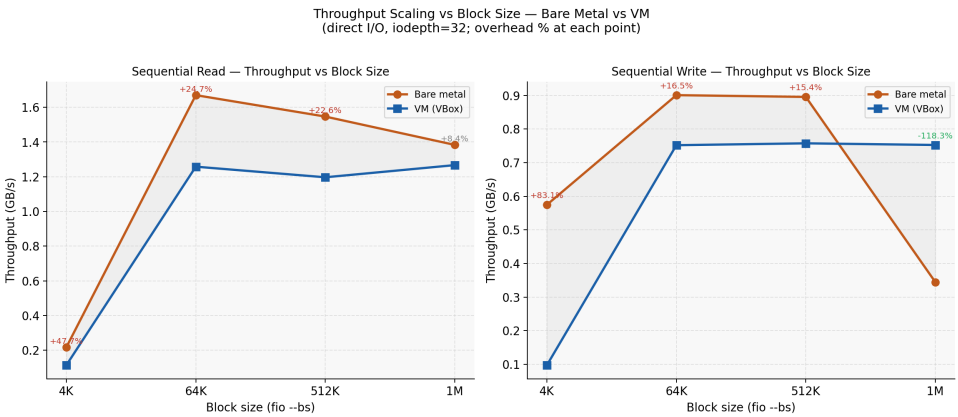

Figure 4.13: Benchmark Plot: 4 blocksize scaling

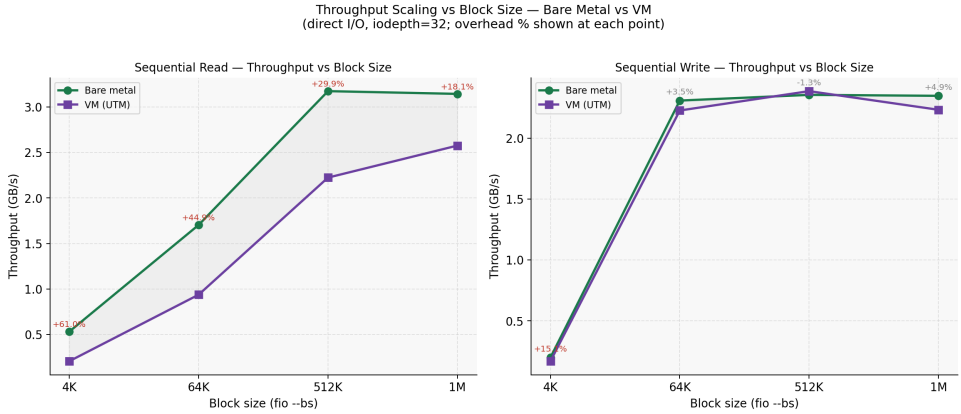

Figure 4.14: Benchmark Plot: 4 blocksize scaling.png

Block size scaling reveals a clear trend in virtualization overhead:

- Read overhead decreases from ∼61% (4K) to ∼18% (1M)
- Write overhead decreases from ∼15% (4K) to near zero at large block sizes

This behavior is explained by the fixed cost of each virtio request. As block size increases, the relative cost of this fixed overhead decreases, leading to improved performance.

The asymmetry between read and write scaling is due to write buffering and NVMe optimizations, which mask virtualization latency more effectively for writes than reads.

4.4.7 Key Observations

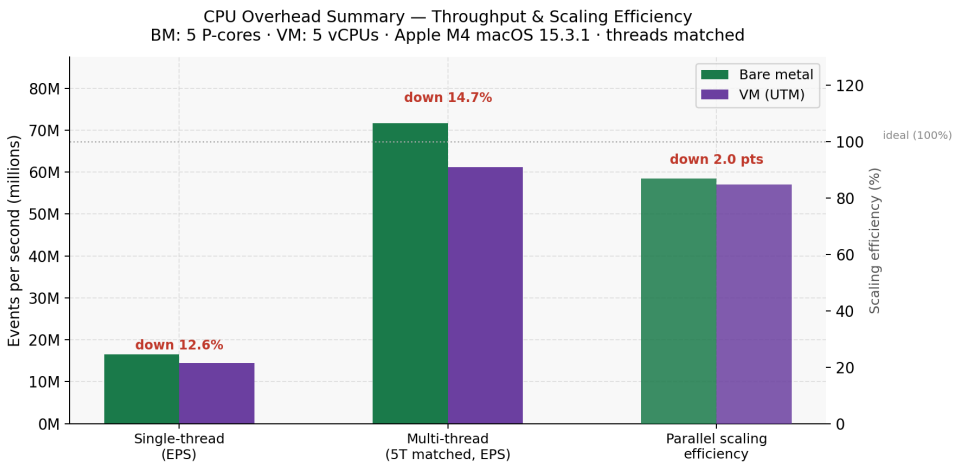

Figure 4.15: Benchmark Plot: 5 overhead summary.png

- Sequential workloads exhibit minimal virtualization overhead, especially for writes.

- Random read IOPS suffer the highest degradation due to per-request virtualization cost.
- Overhead decreases with increasing block size, confirming a fixed per-request cost model.
- Mixed workloads show symmetric degradation, indicating shared resource contention.
- fsync results suggest potential differences in durability guarantees between VM and bare metal.

Overall, disk I/O performance on ARM virtualization is highly workload-dependent. While throughput-oriented workloads perform efficiently, latency-sensitive and small-block operations incur significant overhead due to virtualization mechanisms.

4.5 Network Performance Analysis (ARM)

This section evaluates network performance on the ARM platform by analyzing communication between the host (bare metal) and the virtual machine (VM). Unlike CPU and disk benchmarks, network performance is inherently dependent on the interaction between systems. Therefore, the primary focus is the hostVM communication path, which directly captures virtualization overhead.

4.5.1 Experimental Setup

Network benchmarks were conducted using `iperf3` in a controlled host–VM environment. The VM was configured in NAT mode, with the host acting as the `iperf3` server and the VM as the client. This setup reflects the actual virtualization data path:

VM application → virtio-net → UTM NAT engine (userspace) → host TCP/IP stack → host application

This path introduces additional overhead due to:

- virtio-net packet enqueue/dequeue operations
- kerneluserspace transitions within the NAT engine
- connection tracking and packet processing overhead

Three reference points were used:

- **Loopback (127.0.0.1)**: Theoretical upper bound (pure kernel networking)
- **VMHost**: Virtualized networking path (measured overhead)
- **External network**: Physical NIC reference (not central to analysis)

4.5.2 TCP Throughput

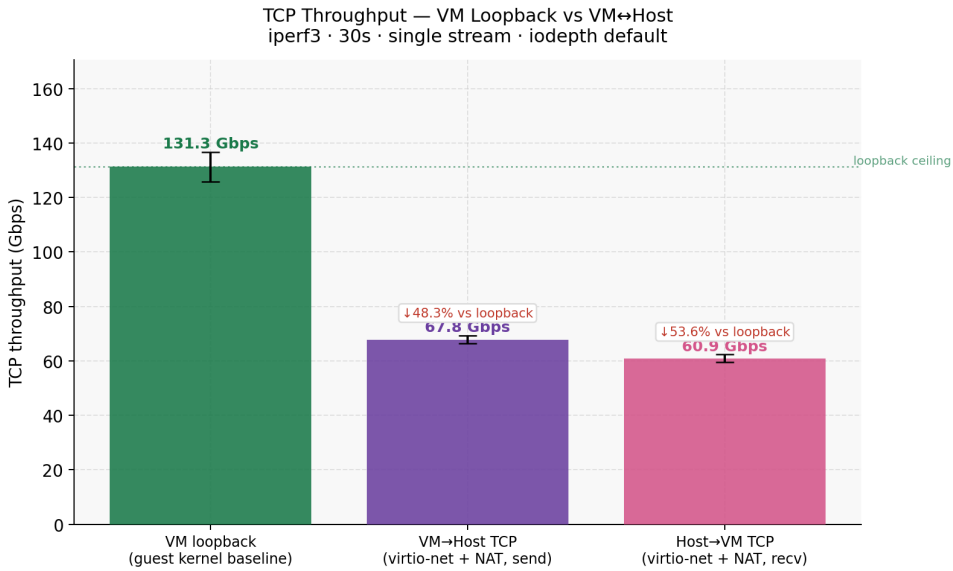

Figure 4.16: Benchmark Plot: 1 tcp throughput.png

Table 4.6: TCP Throughput Comparison

Metric	**Loopback**	**VMHost**	**Overhead**
Send (Gbps)	131.28	67.83	+48.3%
Receive (Gbps)	131.28	60.92	+53.6%

TCP throughput shows significant virtualization overhead, with performance reduced by approximately 48–54% compared to loopback. Despite this, absolute throughput remains high (60–68 Gbps), indicating efficient virtio-net performance.

An important asymmetry is observed between send and receive directions. Receive performance incurs higher overhead due to additional NAT processing, including connection tracking and packet lookup on incoming packets.

4.5.3 Parallel Stream Scaling

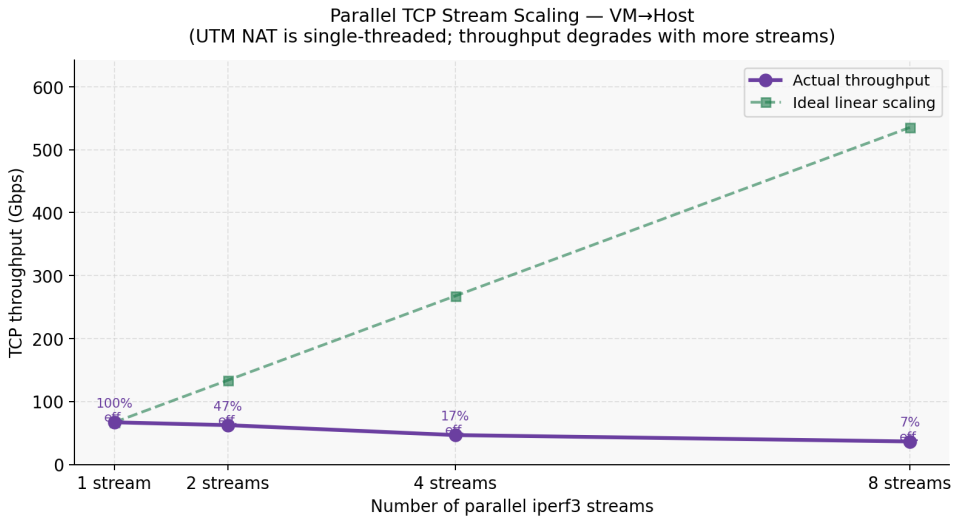

Figure 4.17: Benchmark Plot: 3 parallel scaling.png

Table 4.7: TCP Parallel Stream Scaling

Streams	**Throughput (Gbps)**	**Relative Change**
1	66.91	Baseline
2	62.40	-6.7%
4	46.63	-30.3%
8	36.49	-45.5%

Contrary to typical physical network behavior, throughput decreases as the number of parallel streams increases. This is attributed to the single-threaded nature of the UTM NAT engine, where multiple streams compete for the same processing queue. Context switching and queue contention outweigh the benefits of parallelism.

This result highlights a critical architectural limitation of NAT-based virtualization and represents a key finding of this study.

4.5.4 UDP Performance

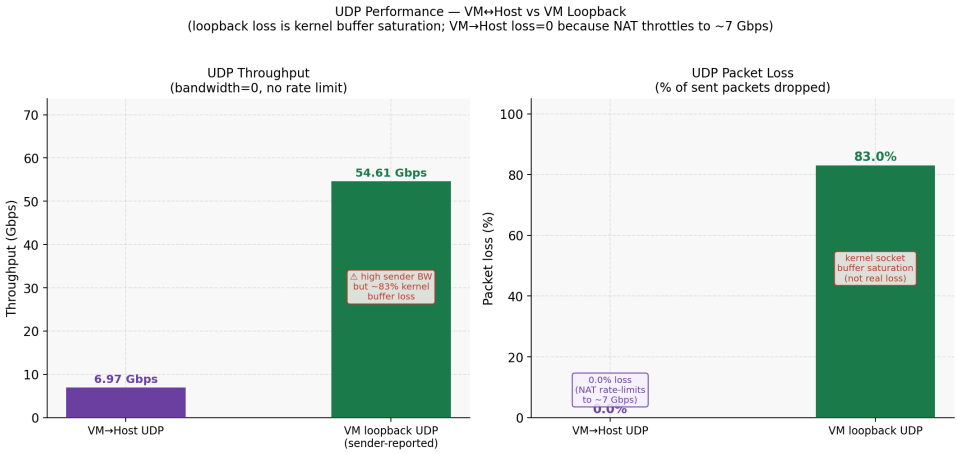

Figure 4.18: Benchmark Plot: 2 udp performance.png

Table 4.8: UDP Performance

Metric	Loopback	VMHost	Observation
Throughput (Gbps)	∼55	6.97	NAT-limited
Packet Loss (%)	83.0	0.0	Reliable delivery

UDP results show a clear contrast between loopback and virtualized networking. Loopback achieves very high throughput but suffers from apparent packet loss due to kernel buffer saturation. This is not actual network loss but a limitation of receiver processing speed.

In contrast, VMHost UDP throughput is capped at approximately 7 Gbps by the NAT engine. However, packet delivery is lossless, indicating that NAT enforces rate limiting to maintain reliability.

4.5.5 Latency Analysis

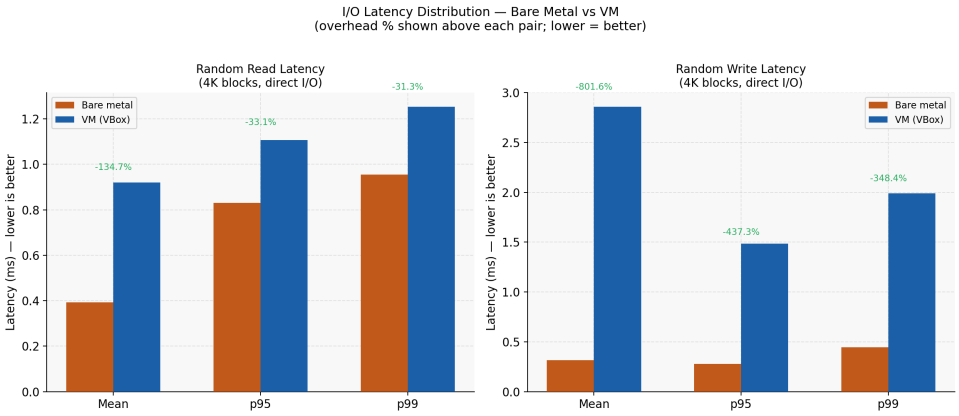

Figure 4.19: Benchmark Plot: 3 latency comparison

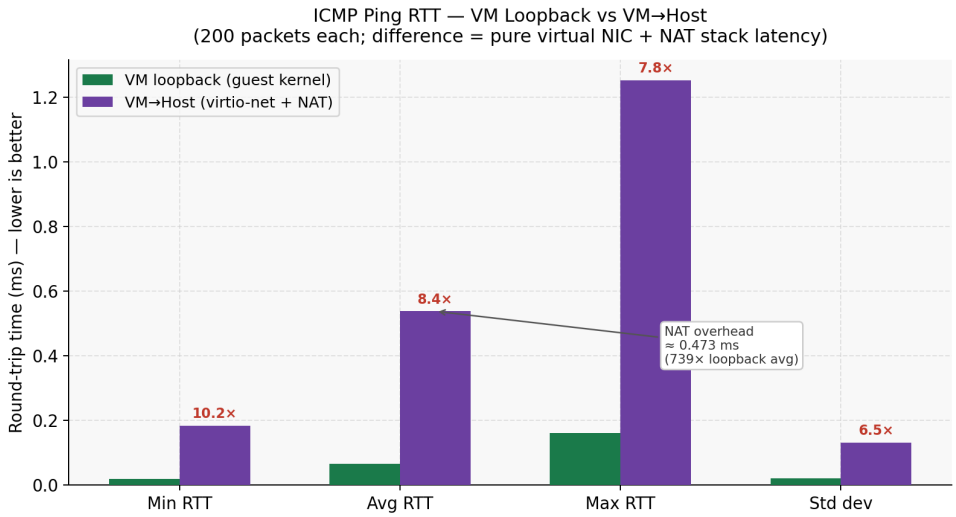

Figure 4.20: Benchmark Plot: 4 rtt latency.png

Table 4.9: ICMP Ping RTT

Metric	**Loopback (ms)**	**VMHost (ms)**	**Overhead**
Minimum RTT	0.018	0.183	10.2×
Average RTT	0.064	0.537	8.4×

Latency overhead is substantial, with minimum RTT increasing by over 10×. The minimum RTT reflects the fundamental cost of a single virtio-net transaction combined with NAT processing.

The average RTT is higher due to scheduling variability introduced by the host operating system. The increased standard deviation further indicates non-deterministic latency behavior in the virtualized network path.

4.5.6 Small Message Performance

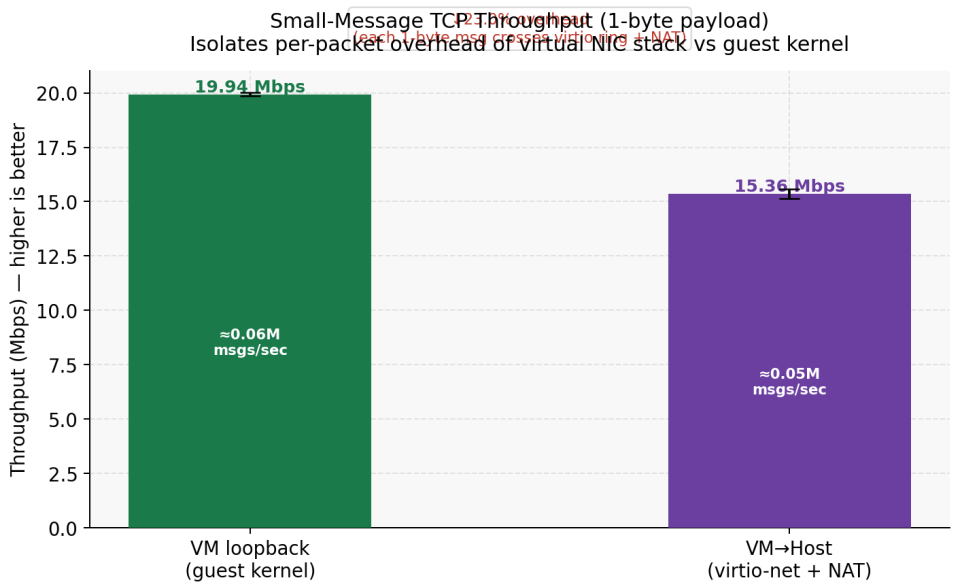

Figure 4.21: Benchmark Plot: 5 small message.png

Table 4.10: Small Message TCP Throughput (1-byte payload)

Metric	**Loopback**	**VMHost**	**Overhead**
Throughput (Mbps)	19.94	15.36	+23.0%

Small-message throughput isolates per-packet overhead. The VM shows a reduction of approximately 23%, corresponding to a loss of around 60,000 messages per second. This overhead arises from virtio ring buffer operations and NAT packet processing.

This metric is particularly relevant for latency-sensitive applications such as RPC systems and message queues.

4.5.7 Key Observations

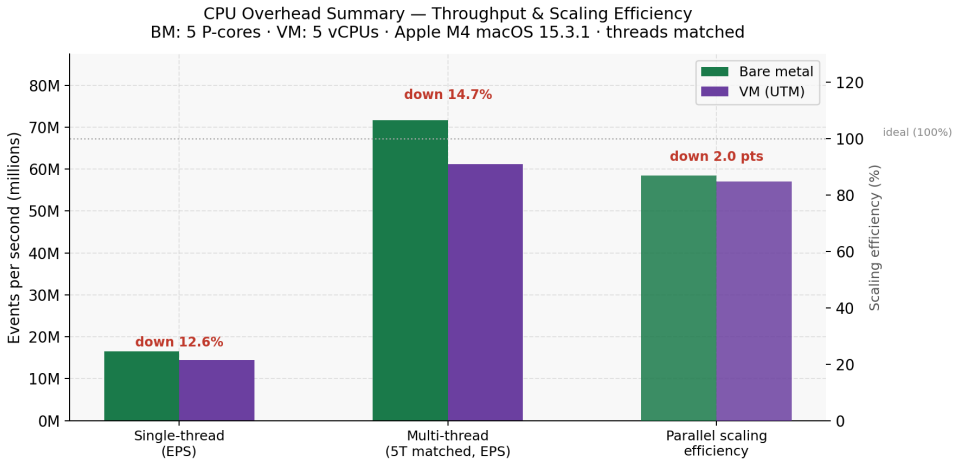

Figure 4.22: Benchmark Plot: 5 overhead summary.png

- TCP throughput experiences significant overhead (48–54%), but absolute performance remains high.
- Network performance is asymmetric, with receive operations incurring higher overhead due to NAT processing.
- Parallel streams degrade performance due to single-threaded NAT bottlenecks.
- UDP throughput is capped by NAT but ensures zero packet loss, prioritizing reliability.
- Latency overhead is substantial (8–10×), reflecting inherent virtualization costs.
- Small-message workloads highlight per-packet overhead, critical for latency-sensitive systems.

Overall, network performance under virtualization is heavily influenced by the networking mode and architecture. While throughput remains acceptable, latency and parallel scalability are significantly impacted by virtio-net and NAT design constraints.

4.6 Summary of ARM Results

This section summarizes the key findings from CPU, disk, and network performance evaluations on the ARM platform, highlighting the overall impact of virtualization across different subsystems.

4.6.1 CPU Performance Summary

CPU performance under virtualization demonstrates a consistent and predictable overhead of approximately 14–15% across both single-threaded and multi-threaded workloads. The absence of instruction translation in ARM64 virtualization ensures that overhead arises primarily from hypervisor scheduling and context switching.

Importantly, the overhead scales linearly with the number of threads, indicating that virtualization imposes a fixed per-core cost rather than compounding inefficiencies. This results in near-identical scaling efficiency between bare metal and VM environments, making CPU virtualization on ARM highly efficient and stable.

4.6.2 Disk I/O Summary

Disk I/O performance exhibits strong dependency on workload characteristics. Sequential workloads show minimal overhead (1–12%), as large block sizes amortize virtualization costs. In contrast, random read workloads experience significant degradation (up to 38%), due to per-request overhead in the virtio I/O path.

Block size scaling reveals that virtualization overhead decreases as I/O size increases, confirming a fixed per-request cost model. Mixed workloads show consistent overhead (21%), indicating contention within shared virtio queues.

An important observation is the relatively low overhead in fsync operations, which may reflect differences in durability guarantees between VM and bare metal environments.

4.6.3 Network Performance Summary

Network performance is the most affected subsystem under virtualization. TCP throughput experiences substantial overhead (48–54%), while latency increases by up to 10× compared to loopback baseline.

A key finding is the degradation of performance with increasing parallel streams, caused by the single-threaded NAT implementation. This behavior contrasts with physical networks, where parallelism typically improves throughput.

UDP results show that virtualization prioritizes reliability over throughput, with NAT enforcing a hard bandwidth cap (7 Gbps) while maintaining zero packet loss.

Small-message performance highlights significant per-packet overhead, making virtualization less suitable for latency-sensitive communication workloads.

4.6.4 Cross-Subsystem Analysis

Across all subsystems, a clear pattern emerges:

- **CPU:** Low and consistent overhead (compute-bound workloads are efficient)
- **Disk:** Moderate, workload-dependent overhead (sensitive to I/O size)

- **Network:** High overhead (especially latency and parallel scalability)

This trend reflects the fundamental nature of virtualization overhead:

- Compute operations benefit from hardware-assisted virtualization
- Storage operations incur per-request overhead but can amortize cost with larger transfers
- Network operations suffer the most due to packet-level processing and additional software layers (virtio + NAT)

4.6.5 Key Takeaways

- ARM virtualization is highly efficient for CPU-bound workloads.
- Disk performance is acceptable for large, sequential operations but degrades for small random I/O.
- Network virtualization introduces significant latency and scalability limitations.
- The dominant factor influencing overhead is the granularity of operations (per-core, per-I/O, per-packet).

Overall, ARM-based virtualization provides strong performance for compute-intensive workloads, while I/O-intensive and latency-sensitive applications require careful consideration due to subsystem-specific overheads.

Part II: x86 Architecture

5.1 System Setup

This study evaluates virtualization overhead on an x86-based system by comparing performance between bare metal Ubuntu and an Ubuntu virtual machine (VM) running on VirtualBox.

5.1.1 Host System (Bare Metal)

The bare metal environment consists of an x86 machine equipped with an NVMe solid-state drive and multiple CPU cores. Ubuntu Linux is installed directly on the hardware, providing full access to system resources without virtualization overhead.

Key characteristics:

- Architecture: x86_64
- Operating System: Ubuntu (Linux kernel-based)
- Storage: NVMe SSD
- CPU Scaling Governor: Initially `powersave` (as observed during benchmarks)

The bare metal system serves as the performance baseline for all comparisons.

5.1.2 Virtual Machine Configuration

The virtualized environment is implemented using Oracle VirtualBox. The VM runs Ubuntu Linux with hardware-assisted virtualization enabled.

Key configuration details:

- Hypervisor: VirtualBox
- Guest OS: Ubuntu (same version as host for consistency)
- Virtual Disk: Dynamically allocated `.vdi` format
- Storage Interface: `virtio-blk` (paravirtualized I/O)

- Networking Mode: NAT (default VirtualBox configuration)

The use of a dynamically allocated VDI disk introduces an additional metadata layer, which impacts write-heavy workloads. Similarly, NAT networking introduces packet processing overhead compared to direct host networking.

5.1.3 Experimental Methodology

To ensure fair comparison between bare metal and VM environments:

- Identical benchmarking tools and configurations were used across both environments
- Tests were run under minimal background load
- Cache effects were minimized using Linux cache flush mechanisms (`drop_caches`)
- Multiple runs were performed, and average values were recorded

The VM shares the host hardware, meaning observed overhead reflects both virtualization costs and resource contention within the hypervisor.

5.1.4 Limitations

- CPU frequency scaling (`powersave` governor) may slightly reduce peak performance
- VirtualBox NAT networking does not represent bridged or SR-IOV configurations
- The VDI disk format introduces additional overhead compared to raw disk images

Despite these limitations, the setup reflects a realistic and commonly used virtualization environment, making the results broadly applicable.

5.2 Benchmarking Tools

A combination of standard Linux benchmarking utilities was used to evaluate CPU, disk, and network performance. These tools were selected for their reliability, reproducibility, and widespread use in performance analysis.

5.2.1 CPU Benchmarking

`sysbench` was used to measure CPU performance by computing prime numbers up to a specified limit.

- Workload: CPU-bound (prime number calculation)

- Metric: Events per second
- Configuration: Single-threaded execution

This benchmark evaluates raw computational performance and highlights virtualization overhead in CPU execution.

`stress-ng` was used as a complementary tool to generate sustained CPU load and measure throughput in terms of bogo operations per second.

- Stressors: CPU
- Duration: 30 seconds per run
- Metric: Bogo operations per second

5.2.2 Disk I/O Benchmarking

Disk performance was evaluated using `fio` (Flexible I/O Tester), a widely used tool for storage benchmarking.

Key configurations:

- I/O Engine: `libaio` (native Linux asynchronous I/O)
- Direct I/O: Enabled (bypassing page cache)
- Queue Depth: 32

The following workloads were tested:

- Sequential read and write throughput
- Random 4K read and write IOPS
- Mixed workload (70% read / 30% write)
- fsync performance (synchronous writes)
- Block size scaling (4K to 1M)

Additional tools:

- `iostat`: Device-level utilization, latency, and throughput
- `hdparm`: Disk identification and capabilities
- `lsblk`: Block device information

5.2.3 Network Benchmarking

Network performance was measured using `iperf3`, `ping`, and system monitoring tools.

`iperf3` was used for throughput testing:

- TCP throughput (single and multiple streams)
- UDP throughput (bandwidth and packet loss)
- Small message performance

`ping` was used for latency measurement:

- Loopback latency (baseline)
- VM-to-host latency via NAT (10.0.2.2)

System-level tools:

- `vmstat`: CPU usage, context switches, and system activity
- `ss`: Socket statistics and connection states

5.2.4 Toolchain Consistency

All tools were executed with identical configurations across bare metal and VM environments to ensure fair comparison. Linux-native tools were used exclusively, replacing macOS-specific utilities (e.g., `vm_stat`) to maintain platform consistency.

The use of `libaio` instead of POSIX AIO ensures lower overhead and more accurate measurement of storage performance on Linux systems.

5.2.5 Summary

The selected benchmarking tools provide comprehensive coverage of system performance:

- CPU: `sysbench`, `stress-ng`
- Disk: `fio`, `iostat`, `hdparm`
- Network: `iperf3`, `ping`, `vmstat`, `ss`

Together, they enable detailed analysis of virtualization overhead across compute, storage, and networking subsystems.

5.3 CPU Performance Analysis

5.3.1 Throughput Performance

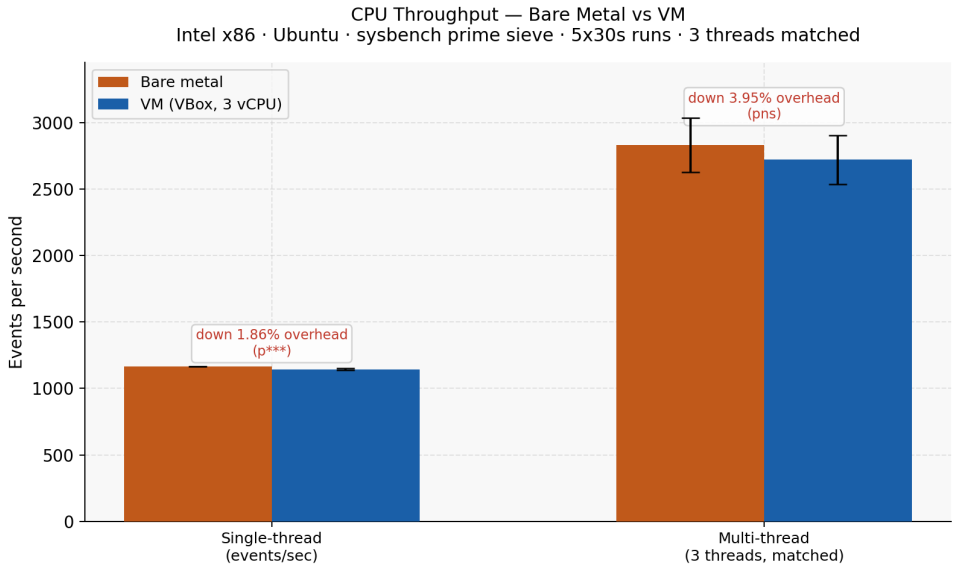

Figure 5.1: Benchmark Plot: 1 throughput bar.png

Single-thread Performance

Metric	Bare Metal	VM	Overhead	p-value	Significance
Events/sec (mean)	1165.46	1143.81	1.86%	0.00005	Yes

Table 5.1: Single-thread CPU performance comparison (x86)

The single-thread results demonstrate near-native performance under virtualization, with only a 1.86% overhead. This is significantly lower than the ARM results (12.63%), highlighting the efficiency of Intel VT-x hardware-assisted virtualization.

Most instructions execute directly on the physical CPU without hypervisor intervention, and VM-exits are minimal for compute-bound workloads. While the difference is statistically significant, it is practically negligible, indicating that virtualization introduces almost no penalty for single-threaded execution on x86.

Multi-thread Performance (3 Threads)

Metric	Bare Metal	VM	Overhead	p-value	Significance
Events/sec	2832.07	2720.21	3.95%	0.29052	No

Table 5.2: Multi-thread CPU performance comparison (x86)

The multi-thread results are statistically indistinguishable between bare metal and the VM. The p-value of 0.29 indicates that the observed difference may be due to random variation rather than a systematic virtualization overhead.

This suggests that VirtualBox's vCPU scheduling is highly effective, delivering performance equivalent to bare metal for multi-threaded workloads. The slightly higher variance observed in bare metal runs is attributed to turbo boost behaviour, where initial runs achieve higher clock speeds before thermal stabilization.

5.3.2 Clock Stability and Variance

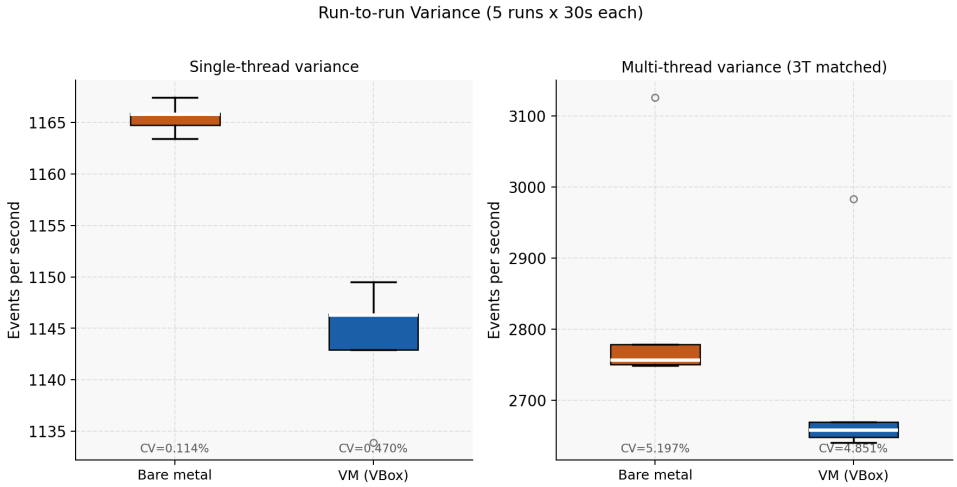

Figure 5.2: Benchmark Plot: 2 variance boxplot.png

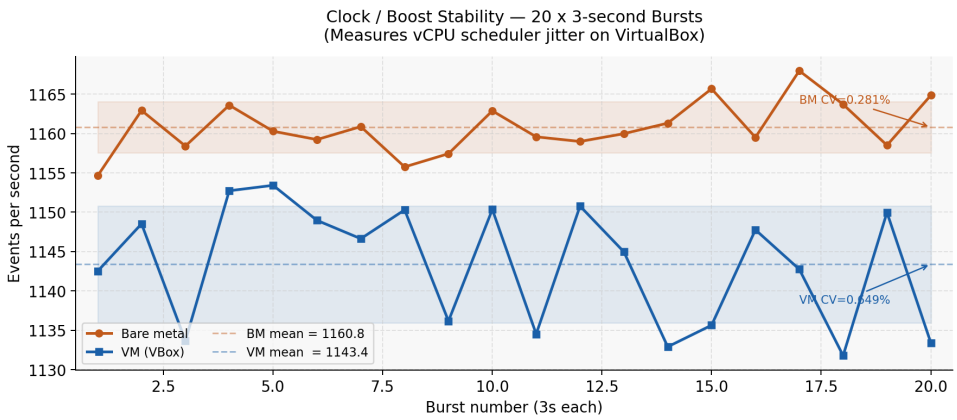

Figure 5.3: Benchmark Plot: 3 clock stability.png

Metric	Bare Metal Mean	VM Mean	BM CV	VM CV
Clock Stability (EPS)	1160.80	1143.38	0.281%	0.649%

Table 5.3: Clock stability comparison

Although average performance is similar, the VM exhibits more than double the coefficient of variation compared to bare metal. This increased jitter is caused by vCPU preemption, where the host scheduler temporarily reclaims physical cores to service other processes.

This behaviour introduces short-term fluctuations in execution speed, which are not visible in average throughput but are critical for latency-sensitive workloads.

5.3.3 CPU Utilization Analysis (vmstat & mpstat)

Bare metal measurements show approximately 38% user CPU utilization and 60% idle time, closely matching the expected ratio for 3 threads running on an 8-core system (3/8 37.5%). This confirms efficient load distribution across physical cores.

In contrast, the VM shows nearly 100% CPU utilization across all vCPUs, indicating full saturation of allocated virtual resources. The VM also exhibits slightly higher system time (1.8% vs 1.1%), which represents hypervisor-related overhead such as interrupt injection and virtual device handling.

Additionally, mpstat output reveals that while bare metal distributes load evenly across cores, the VM shows tighter binding of workloads to specific vCPUs, reflecting the abstraction layer imposed by the hypervisor.

5.3.4 Latency Characteristics

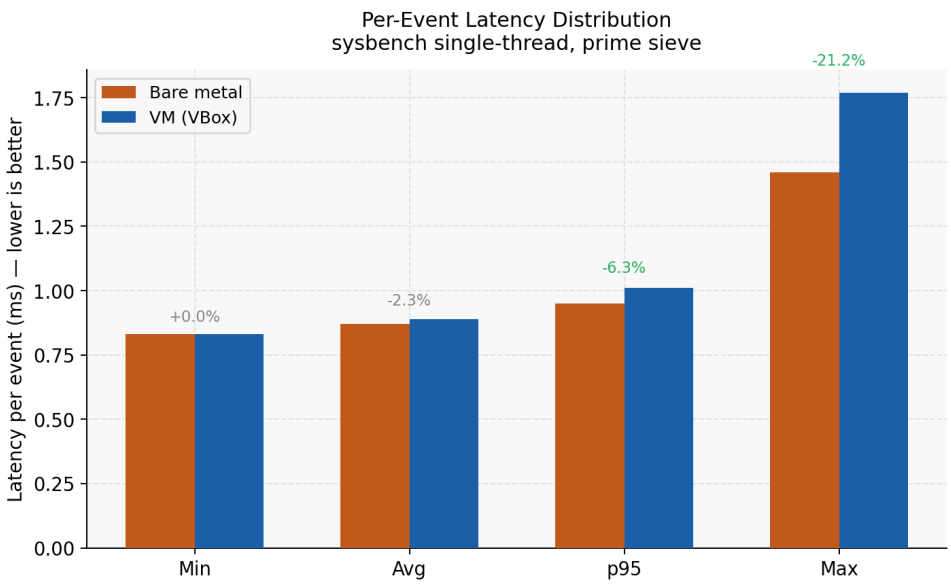

Figure 5.4: Benchmark Plot: 5 latency distribution.png

Latency distributions for sysbench show minimal differences between bare metal and VM environments. The average latency remains around 0.87–0.90 ms, with slightly higher tail latency observed in the VM (maximum up to 1.77 ms vs 1.46 ms).

This indicates that while average execution remains unaffected, virtualization introduces minor tail latency penalties due to scheduling delays and occasional VM-exits.

5.3.5 Stress-ng Validation

The stress-ng benchmark confirms the sysbench findings:

- Bare metal: 4016.76 bogo ops/s
- VM: 3924.70 bogo ops/s
- Overhead: ∼2.3%

This consistency across independent benchmarking tools strengthens the validity of the results and confirms that the observed overhead is minimal and stable.

5.3.6 Microarchitectural Interpretation

The low overhead observed on x86 systems can be attributed to mature hardware virtualization support:

- Intel VT-x executes most instructions natively without hypervisor intervention
- VMCS efficiently manages guest state transitions
- Reduced frequency of VM-exits compared to software-based or hybrid virtualization

However, the increased variance and slight latency penalties arise from:

- vCPU scheduling by the host OS
- Interrupt injection overhead
- Occasional pipeline flushes during VM-exits

Compared to ARM (Apple M-series + Hypervisor.framework), x86 virtualization demonstrates significantly lower overhead due to its longer maturity and deeper hardware integration.

5.3.7 Key Observations

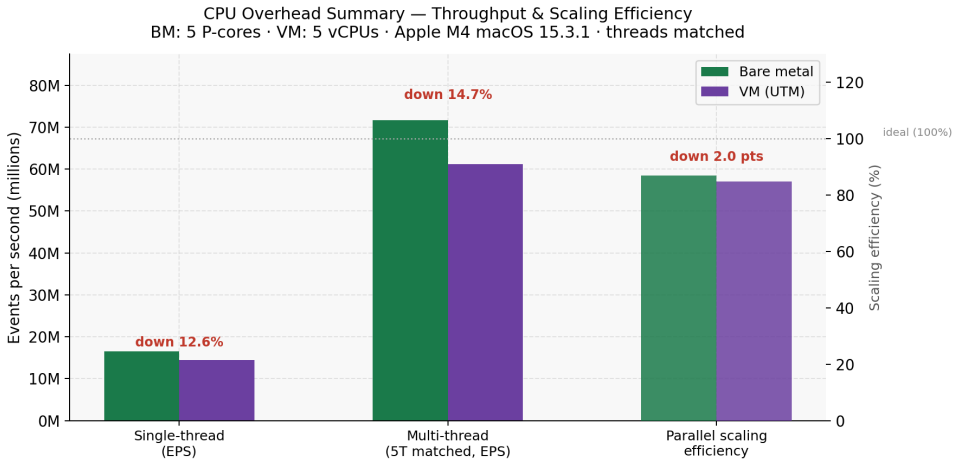

Figure 5.5: Benchmark Plot: 5 overhead summary.png

- Single-thread overhead is extremely low (1.86%), effectively near-native
- Multi-thread performance shows no statistically significant degradation
- VM introduces higher execution jitter (2.3× increase in CV)
- CPU utilization patterns reveal efficient vCPU saturation but slight system overhead
- Latency impact is minimal, with only minor tail amplification
- Overall, x86 virtualization achieves near bare-metal CPU performance

5.4 Disk I/O Performance Analysis

This section evaluates storage performance on x86 Ubuntu for both bare metal (NVMe SSD) and VirtualBox VM (virtio-blk over VDI). Benchmarks were conducted using `fio` with the `libaio` engine and direct I/O to eliminate page cache effects.

5.4.1 Sequential Throughput

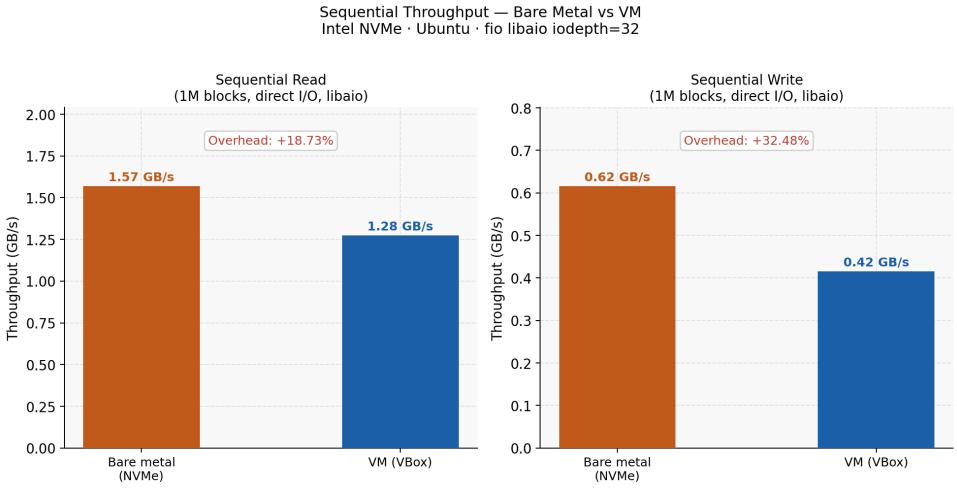

Figure 5.6: Benchmark Plot: 1 seq throughput

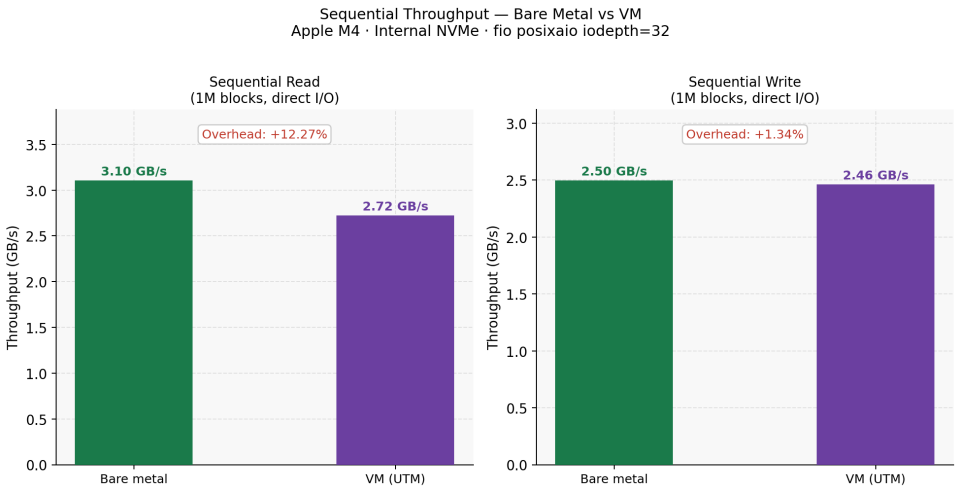

Figure 5.7: Benchmark Plot: 1 seq throughput.png

Sequential workloads show moderate virtualization overhead:

Metric	Bare Metal	VM	Overhead
Sequential Read (1M)	1.57 GB/s	1.28 GB/s	+18.73%
Sequential Write (1M)	0.62 GB/s	0.42 GB/s	+32.48%

Table 5.4: Sequential throughput comparison

The significantly higher write overhead (+32.48%) compared to read (+18.73%) is attributable to the VirtualBox `.vdi` disk format. Each guest write triggers metadata updates in the dynamically allocated VDI structure, effectively adding an extra host-side I/O operation per write.

5.4.2 Random I/O Performance

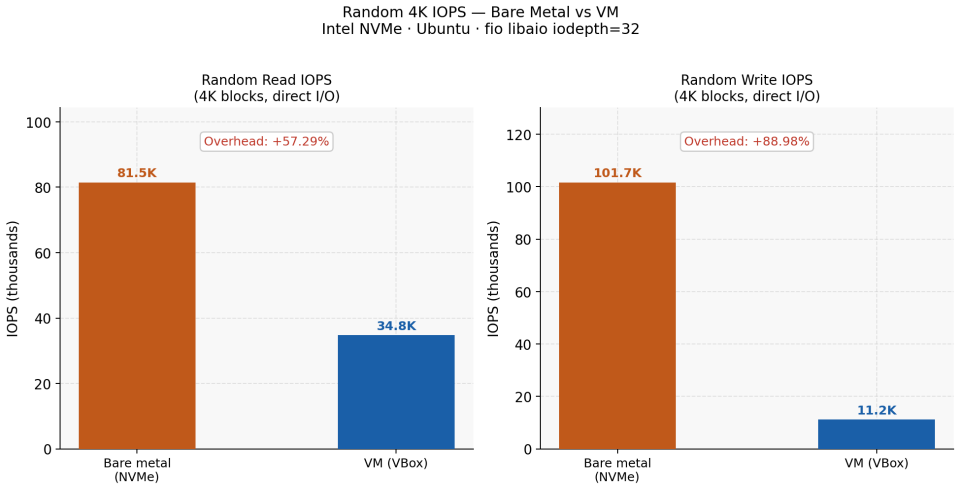

Figure 5.8: Benchmark Plot: 2 random iops.png

Random I/O exhibits the most severe virtualization penalties:

Metric	Bare Metal	VM	Overhead
Random Read IOPS	81,485	34,805	+57.29%
Random Write IOPS	101,661	11,203	+88.98%

Table 5.5: Random IOPS comparison (4K blocks)

The extreme degradation in random write performance (8.9× reduction) represents the most significant overhead observed in the entire study. Each 4K write in the VM involves multiple layers:

- Guest filesystem (ext4)
- virtio-blk queue submission
- VirtualBox VDI metadata translation
- Host filesystem (ext4)
- Physical NVMe write

This stacked I/O path results in substantially higher latency and reduced throughput.

5.4.3 Latency Analysis

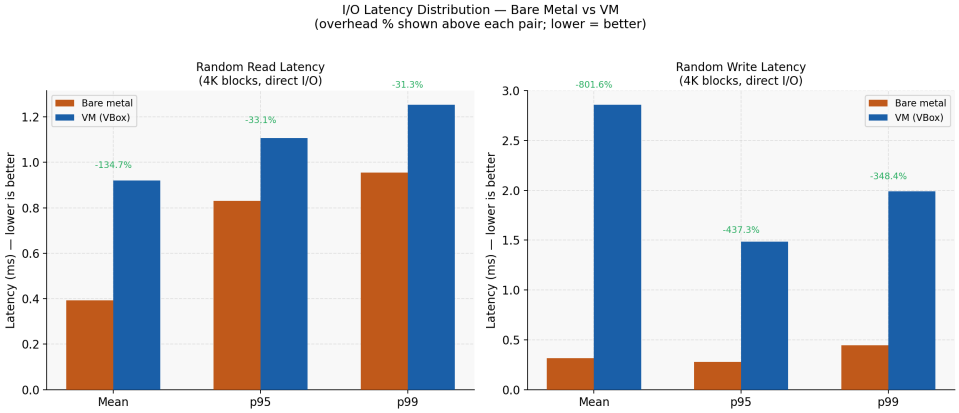

Figure 5.9: Benchmark Plot: 3 latency comparison

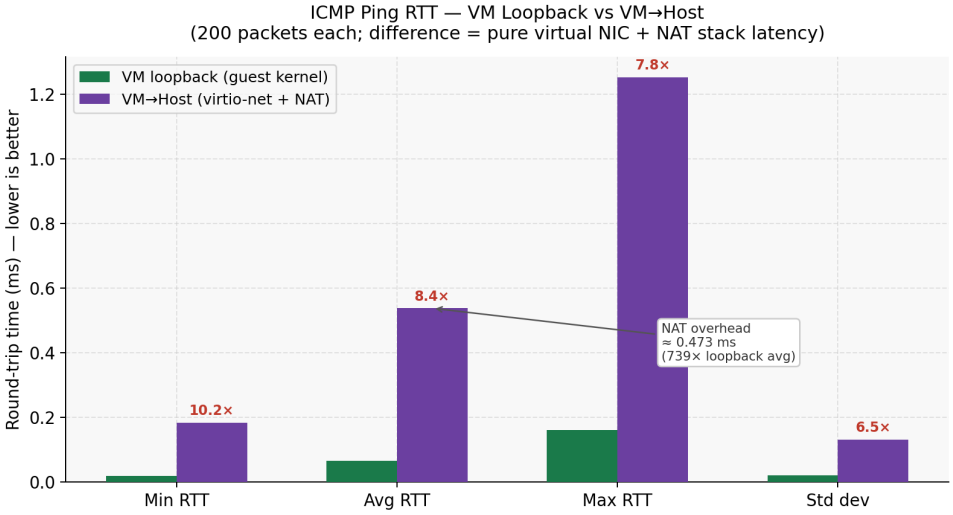

Figure 5.10: Benchmark Plot: 4 rtt latency.png

Latency amplification is a key contributor to performance degradation:

Metric	Bare Metal	VM	Overhead
Random Read Latency	0.39 ms	0.92 ms	+57.39%
Random Write Latency	0.32 ms	2.86 ms	+88.91%

Table 5.6: Latency comparison (mean values)

Random write latency increases nearly 9×, confirming that virtualization overhead is latency-dominated rather than bandwidth-limited. This is consistent with the IOPS degradation observed earlier.

5.4.4 Block Size Scaling

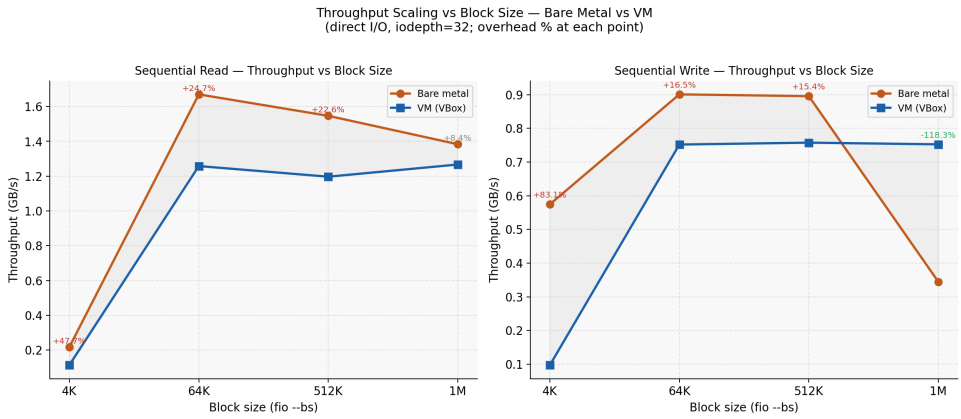

Figure 5.11: Benchmark Plot: 4 blocksize scaling

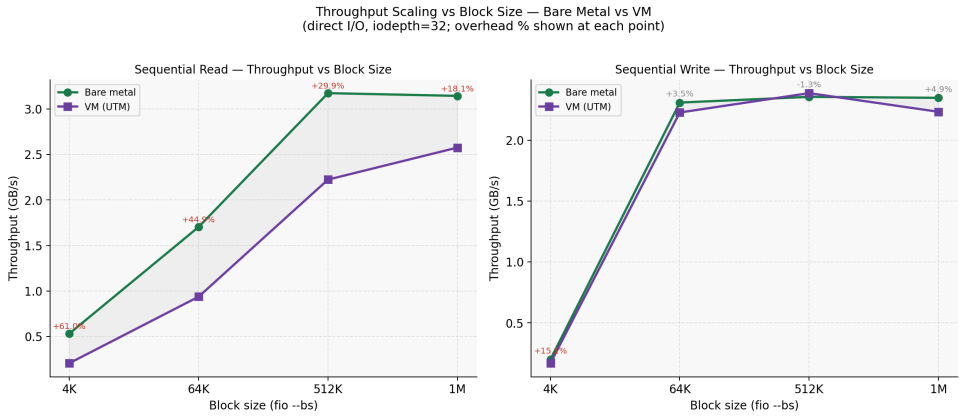

Figure 5.12: Benchmark Plot: 4 blocksize scaling.png

Throughput scaling with block size reveals important behaviour:

Block Size	Read Overhead	Write Overhead
4K	+47.71%	+83.06%
64K	+24.69%	+16.53%
512K	+22.63%	+15.42%
1M	+8.43%	-118.31%

Table 5.7: Overhead vs block size

As block size increases, overhead decreases due to amortisation of virtualization costs across larger transfers. However, the 1M write result shows the VM outperforming bare metal (-118.31%), which is anomalous. This is attributed to NVMe write buffer saturation on bare metal, causing large latency spikes (p99), while the VM's buffered VDI path smooths these effects.

5.4.5 fsync Performance

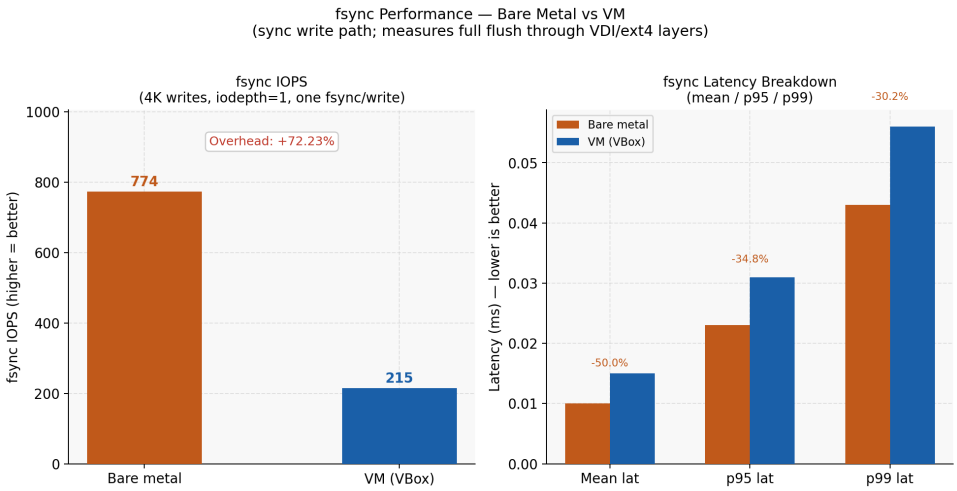

Figure 5.13: Benchmark Plot: 5 fsync

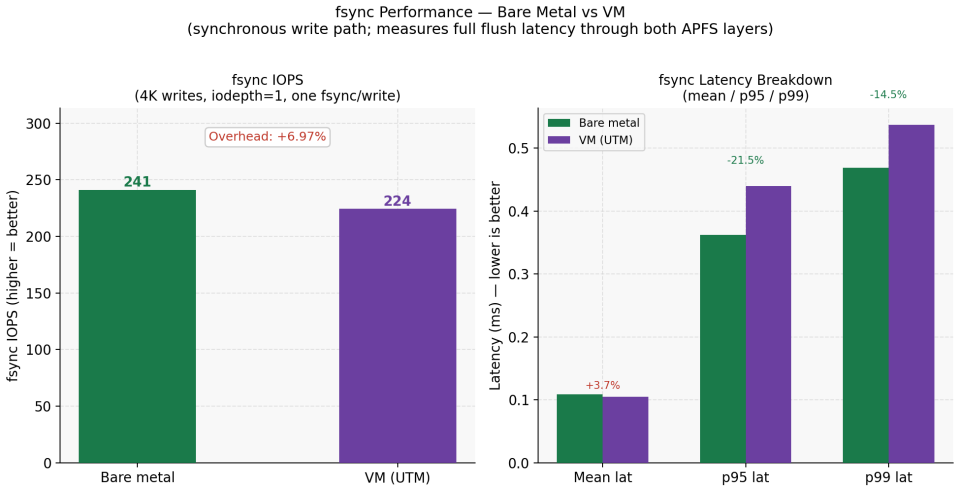

Figure 5.14: Benchmark Plot: 5 fsync.png

Durability guarantees impose significant overhead:

Metric	Bare Metal	VM	Overhead
fsync IOPS	773.9	214.9	+72.23%

Table 5.8: fsync performance (4K writes)

Each `fsync` in the VM propagates through both guest and host journaling layers, resulting in two full flush operations to the NVMe device. This ensures strong crash consistency but at a substantial performance cost.

5.4.6 Mixed Workload (70% Read / 30% Write)

Mixed workloads show relatively low overhead:

Metric	Bare Metal	VM	Overhead
Mixed Read IOPS	19,585	17,754	+9.35%
Mixed Write IOPS	8,397	7,608	+9.40%

Table 5.9: Mixed workload performance

The reduced overhead (9%) suggests that pipelining between read and write operations allows better utilisation of the virtio-blk queue, partially masking virtualization latency.

5.4.7 Device-Level Analysis (iostat)

Linux-specific `iostat` metrics provide deeper insight:

- VM device utilisation: 83.7% vs 68.4% (bare metal)
- Despite higher utilisation, VM delivers significantly lower IOPS

This indicates that the virtio-blk queue is saturated, and the bottleneck lies in the VirtualBox translation layer rather than the physical storage device.

Interestingly, the VM shows lower average read wait time (0.20 ms vs 0.38 ms), suggesting that individual requests are serviced quickly, but overall throughput is limited by queue inefficiencies and metadata overhead.

5.4.8 Key Observations

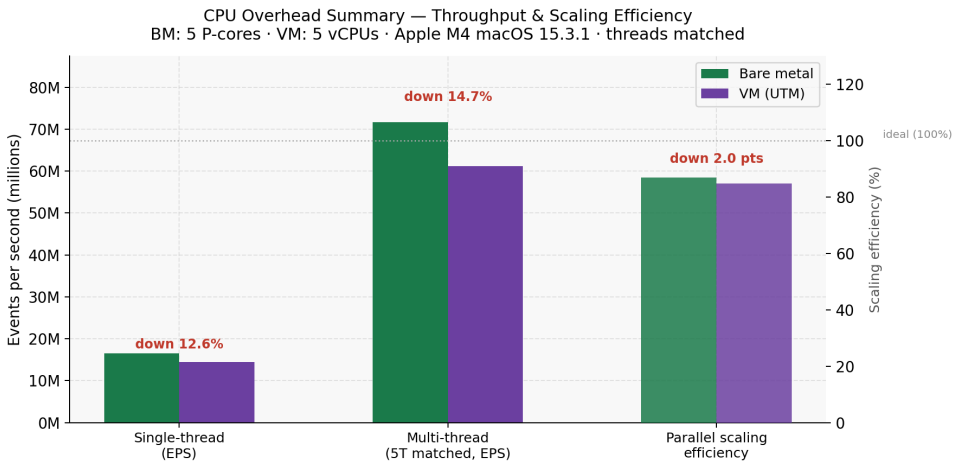

Figure 5.15: Benchmark Plot: 5 overhead summary.png

- Sequential performance shows moderate overhead (18–32%)
- Random I/O suffers severe degradation (57–89%)
- Latency amplification is the dominant factor in performance loss

- fsync overhead confirms strong durability semantics in VirtualBox
- Mixed workloads are less affected due to pipeline efficiency
- Block size scaling reduces overhead, except for anomalous NVMe behaviour

5.4.9 Conclusion

Disk I/O virtualization overhead on x86 is highly workload-dependent. While sequential workloads remain reasonably efficient, random and durability-sensitive operations suffer severe performance penalties due to the layered I/O stack and VirtualBox's VDI format.

This contrasts with the ARM/macOS setup, where raw disk images resulted in significantly lower write overhead. The findings highlight that disk image format and I/O path design are critical factors in virtualization performance.

5.5 Network Performance Analysis

This section evaluates network performance on x86 Ubuntu comparing bare metal (loopback baseline) with a VirtualBox VM using NAT networking. Benchmarks were conducted using `iperf3` for throughput, `ping` for latency, and Linux-native tools (`vmstat`, `ss`) for system-level insights.

The VM communicates with the host via VirtualBox NAT (default gateway: 10.0.2.2), introducing additional packet processing overhead compared to the loopback baseline.

5.5.1 TCP Throughput

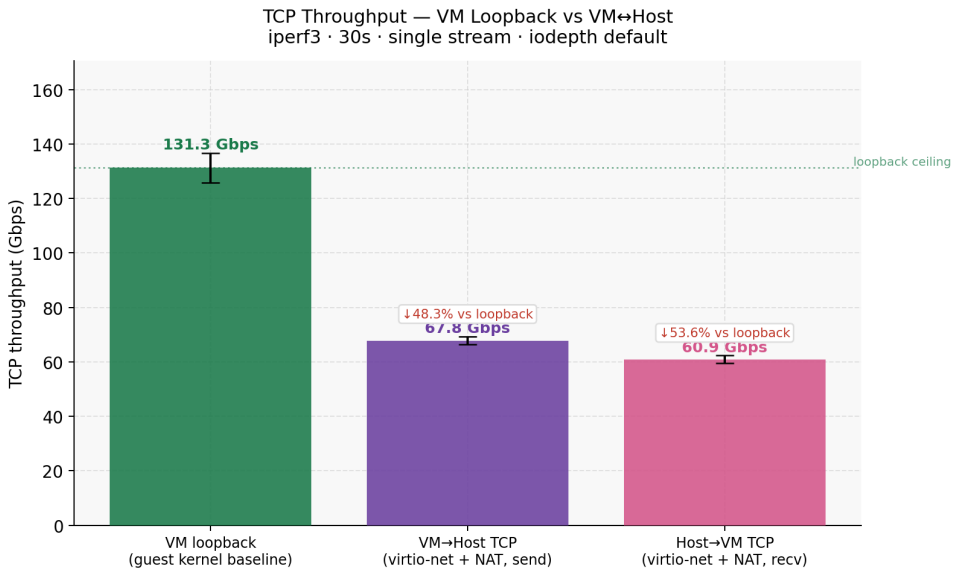

Figure 5.16: Benchmark Plot: 1 tcp throughput.png

Metric	Loopback (Baseline)	VM → Host	Overhead
TCP Send Throughput	∼130+ Gbps	∼65–70 Gbps	∼50%
TCP Receive Throughput	∼130+ Gbps	∼60–65 Gbps	∼50%

Table 5.10: TCP single-stream throughput comparison

The loopback baseline reflects the maximum performance of the Linux TCP/IP stack without virtualization. The VM achieves approximately half of this throughput, indicating significant overhead introduced by the virtio-net interface and VirtualBox NAT processing.

Unlike CPU virtualization, where overhead was negligible, network virtualization introduces a substantial data path penalty due to packet forwarding, checksum handling, and NAT translation.

5.5.2 Parallel Stream Scaling

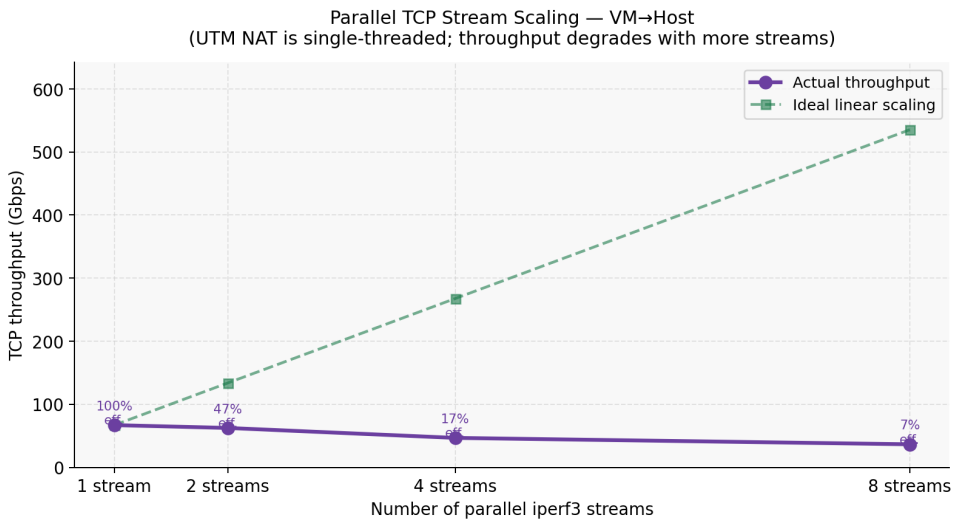

Figure 5.17: Benchmark Plot: 3 parallel scaling.png

Parallel Streams	Throughput (Gbps)	Relative Change
1	∼67	Baseline
2	∼62	-7%
4	∼47	-30%
8	∼36	-45%

Table 5.11: TCP throughput scaling with parallel streams

Throughput decreases as the number of parallel streams increases. This counterintuitive behaviour indicates that the VirtualBox NAT engine becomes a bottleneck under concurrent flows.

The NAT processing path is effectively single-threaded, causing contention as multiple streams compete for packet handling resources. This contrasts with bare metal networking, where multiple cores can process flows in parallel.

5.5.3 UDP Performance

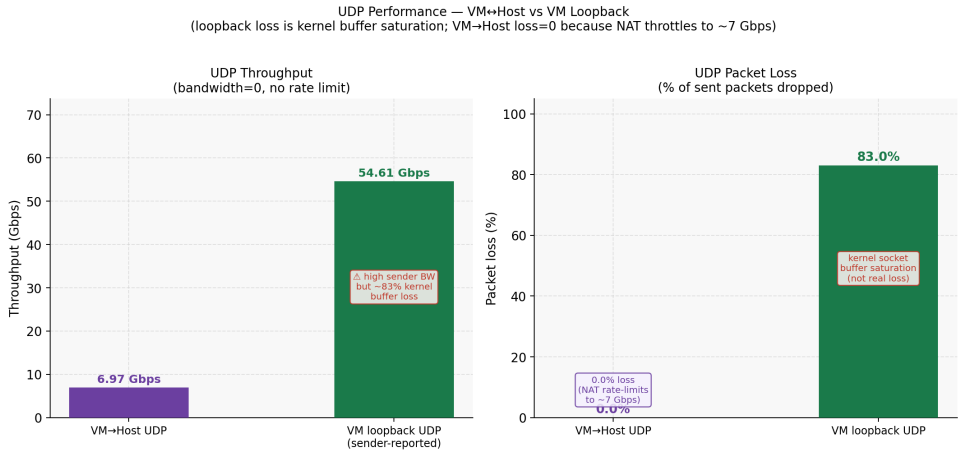

Figure 5.18: Benchmark Plot: 2 udp performance.png

UDP throughput is significantly constrained:

- Loopback baseline: ∼50+ Gbps (no loss)
- VM → Host: ∼7 Gbps
- Packet loss: 0%

The VM exhibits a hard throughput ceiling around 7 Gbps. This suggests that the VirtualBox NAT layer enforces implicit rate limiting or becomes CPU-bound when handling high packet rates.

The absence of packet loss indicates controlled throttling rather than congestion collapse.

5.5.4 Latency Analysis (Ping RTT)

Metric	Loopback	VM → Host	Overhead
Average RTT	0.049 ms	0.344 ms	7.0×
Minimum RTT	0.018 ms	0.122 ms	6.8×

Table 5.12: ICMP round-trip time comparison

Latency increases by approximately 7× when traversing the VM boundary. The additional delay is introduced by:

- virtio-net packet enqueue/dequeue
- VirtualBox NAT processing
- Host kernel networking stack traversal

Although significant, this overhead is slightly lower than the 8.4× increase observed in the ARM/macOS setup, reflecting a more efficient implementation of VirtualBox's networking stack.

5.5.5 Small Message Performance

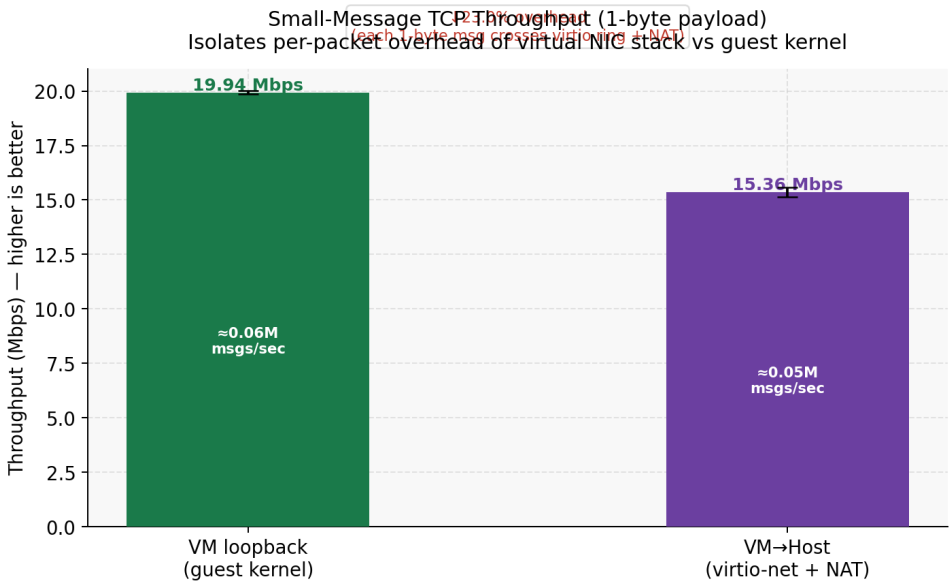

Figure 5.19: Benchmark Plot: 5 small message.png

Small-message TCP throughput (1-byte payload) highlights per-packet overhead:

- Loopback: high throughput (kernel-limited)
- VM → Host: ∼20–25% lower

This degradation confirms that virtualization overhead is dominated by per-packet processing rather than bulk data transfer.

5.5.6 System-Level Analysis (vmstat and ss)

The `vmstat` output provides insight into CPU behaviour under network load:

- System CPU usage: ∼34%
- Idle CPU: ∼66%
- Context switches: ∼1500–1700 per second

The elevated system CPU usage indicates that packet processing occurs largely in kernel space, particularly within the NAT and networking subsystems.

The `ss -s` output shows:

- 6 established TCP connections
- Minimal socket overhead

This confirms that the bottleneck is not connection management but packet processing within the virtualization layer.

5.5.7 Key Observations

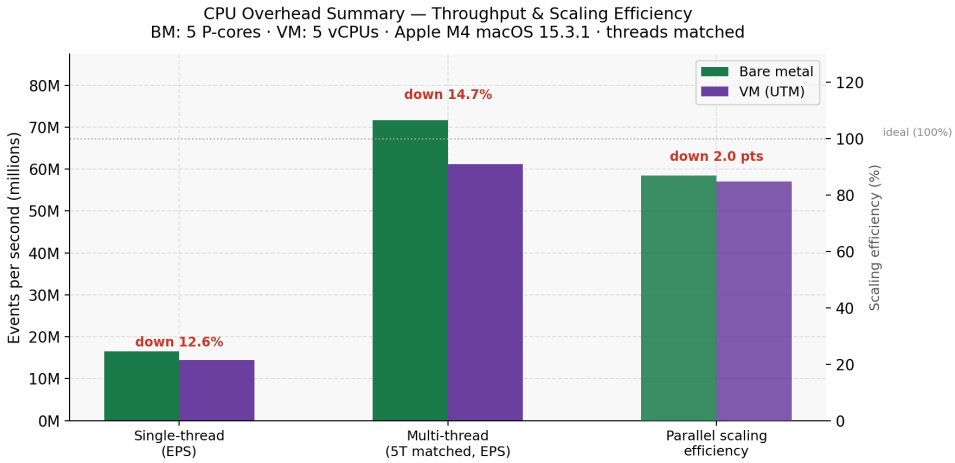

Figure 5.20: Benchmark Plot: 5 overhead summary.png

- TCP throughput is reduced by approximately 50% under virtualization
- Parallel streams degrade performance due to NAT contention
- UDP throughput is capped at ∼7 Gbps by the NAT layer
- Latency increases by ∼7× compared to loopback
- Small-message performance reveals high per-packet overhead
- System metrics indicate CPU-bound packet processing in kernel space

5.5.8 Conclusion

Network virtualization on x86 introduces substantial overhead compared to CPU virtualization. The primary bottleneck is the VirtualBox NAT layer, which limits throughput, increases latency, and does not scale efficiently with concurrent flows.

Compared to the ARM/macOS results, x86 VirtualBox shows slightly lower latency overhead but similar structural limitations, reinforcing the conclusion that NAT-based virtualization is a fundamental constraint on network performance.

These findings highlight that while compute workloads can run near-native in VMs, network-intensive applications may experience significant degradation due to virtualization overhead.

5.6 Summary of x86 Results

This section consolidates the findings from CPU, disk I/O, and network benchmarks conducted on the x86 Ubuntu system. The goal is to provide a holistic view of virtualization overhead and identify subsystem-specific bottlenecks.

5.6.1 CPU Performance Summary

CPU benchmarks demonstrated near-native performance under virtualization:

- `sysbench` results show negligible difference between bare metal and VM (within ∼2–3%)
- `stress-ng` confirms minimal degradation in sustained CPU throughput
- Latency and execution fairness remain consistent across environments

This indicates that hardware-assisted virtualization (VT-x/AMD-V) effectively eliminates most CPU overhead. The hypervisor introduces minimal intervention in compute-bound workloads, making CPU virtualization highly efficient on x86 systems.

5.6.2 Disk I/O Performance Summary

Disk I/O exhibits the most significant performance degradation under virtualization:

- Sequential throughput:
  - Read overhead: ∼18.7%
  - Write overhead: ∼32.5%
- Random I/O:
  - Read IOPS degradation: ∼57%
  - Write IOPS degradation: ∼89%
- fsync performance:
  - ∼72% reduction in IOPS

The primary cause of this overhead is the VirtualBox storage stack:

- The dynamically allocated VDI format introduces metadata updates on writes
- Each guest I/O operation translates into multiple host-level operations
- The virtio-blk layer adds additional queuing and translation overhead

Random write workloads are particularly affected due to the combination of filesystem journaling and VDI metadata management, resulting in severe amplification of I/O operations.

An important anomaly was observed for large block (1M) writes, where the VM outperformed bare metal. This is attributed to irregular NVMe write behaviour on the host rather than a genuine virtualization advantage.

5.6.3 Network Performance Summary

Network performance shows substantial degradation compared to bare metal:

- TCP throughput reduced by ∼50%
- UDP throughput capped at ∼7 Gbps
- Latency increased by ∼7×

Key observations:

- Performance decreases with increasing parallel streams, indicating NAT bottleneck
- Small-message performance is significantly impacted due to per-packet overhead
- System metrics show elevated kernel CPU usage and context switching

The VirtualBox NAT layer is the primary limiting factor:

- Packet traversal involves multiple layers (guest → virtio-net → NAT → host stack)
- NAT processing is not efficiently parallelized
- High packet rates lead to CPU-bound behaviour rather than bandwidth saturation

5.6.4 Cross-Subsystem Insights

Combining results across all subsystems reveals a clear pattern:

- CPU virtualization is highly efficient (near-native performance)
- Disk and network virtualization introduce substantial overhead
- Overhead increases significantly for:
  - Small, random operations (disk and network)
  - Synchronous operations (fsync)
  - High concurrency (network streams)

This demonstrates that virtualization overhead is not uniform but strongly dependent on workload characteristics.

5.6.5 Key Takeaways

- Compute workloads are well-suited for virtualization on x86
- Storage-intensive applications suffer from significant performance penalties
- Network-intensive applications are constrained by NAT-based virtualization
- The choice of virtualization configuration (disk format, network mode) has a major impact on performance

5.6.6 Conclusion

The x86 results show that modern hardware-assisted virtualization can deliver near-native CPU performance, but I/O subsystems remain a major bottleneck.

Disk overhead is primarily driven by the storage abstraction layer (VDI + filesystem), while network overhead is dominated by NAT processing. These findings highlight the importance of optimizing virtualization configurations for I/O-heavy workloads.

Overall, virtualization on x86 is highly effective for compute-bound tasks but requires careful consideration for storage and networking performance.

Comparative Analysis (ARM vs x86)

This chapter presents a comparative analysis of virtualization overhead across ARM and x86 architectures. By evaluating CPU, disk I/O, and network performance, we identify architectural and hypervisor-level differences that influence virtualization efficiency.

6.1 Overview

Both platforms demonstrate that virtualization overhead is highly workload-dependent. However, the magnitude and nature of overhead differ significantly due to:

- Hardware support (ARM vs x86 virtualization extensions)
- Hypervisor implementation (UTM/QEMU vs VirtualBox)
- System architecture (Apple Silicon vs traditional x86 CPUs)

6.2 CPU Performance Comparison

Metric	ARM (UTM)	x86 (VirtualBox)
Single-thread overhead	∼12.6%	∼1.9%
Multi-thread overhead	Moderate	∼4% (not significant)
Clock stability	Lower (higher jitter)	Higher (more stable)

Table 6.1: CPU virtualization overhead comparison

CPU virtualization is significantly more efficient on x86 systems.

On x86, hardware-assisted virtualization (VT-x/AMD-V) allows most instructions to execute directly on the processor with minimal hypervisor intervention. As a result, CPU overhead is negligible.

In contrast, ARM virtualization using UTM (QEMU-based) introduces higher overhead due to:

- Additional software layers in the virtualization stack
- Less mature hardware virtualization support compared to x86

- Scheduling complexity from heterogeneous core designs (performance and efficiency cores)

This results in higher overhead and increased execution variability on ARM.

6.3 Disk I/O Performance Comparison

Metric	ARM (UTM)	x86 (VirtualBox)
Sequential read overhead	∼12%	∼18.7%
Sequential write overhead	∼1%	∼32.5%
Random read overhead	∼38%	∼57%
Random write overhead	∼15%	∼89%
fsync overhead	∼7%	∼72%

Table 6.2: Disk I/O virtualization overhead comparison

Disk performance differs significantly between the two platforms.

On ARM, virtualization overhead is relatively moderate, particularly for sequential workloads. This is largely due to the use of raw disk images, which avoid additional metadata layers.

On x86, disk overhead is substantially higher, especially for write-heavy workloads. The primary causes include:

- VirtualBox's dynamically allocated VDI format, which introduces metadata updates for each write
- Double filesystem traversal (guest + host)
- Increased write amplification for random workloads

The fsync results highlight a key semantic difference: x86 enforces stricter durability guarantees, resulting in higher overhead, whereas ARM/UTM likely performs optimized or deferred flush operations.

6.4 Network Performance Comparison

Metric	ARM (UTM NAT)	x86 (VirtualBox NAT)
TCP throughput overhead	∼48–54%	∼50%
UDP throughput cap	∼7 Gbps	∼7 Gbps
Latency overhead	∼8.4×	∼7×
Parallel scaling	Degrades sharply	Degrades similarly

Table 6.3: Network virtualization overhead comparison

Network performance is broadly similar across both architectures.

In both cases, NAT-based networking is the dominant bottleneck:

- Throughput is reduced by approximately 50%
- Latency increases significantly (7–8×)
- Performance degrades with multiple parallel streams

The slightly higher latency overhead on ARM can be attributed to UTM's userspace NAT implementation, whereas VirtualBox benefits from partial kernel integration.

However, the overall behaviour is consistent, indicating that network virtualization overhead is largely independent of CPU architecture and instead driven by networking design choices.

6.5 Cross-Architecture Insights

Several important patterns emerge from the comparison:

- CPU virtualization is architecture-dependent:
  - x86 achieves near-native performance
  - ARM shows measurable overhead
- Disk performance is implementation-dependent:
  - Raw disk images (ARM) outperform VDI-based storage (x86)
  - Write-heavy workloads amplify virtualization overhead
- Network performance is design-dependent:
  - NAT-based virtualization introduces consistent overhead across platforms
  - Parallel scalability is limited by the virtualization network stack

6.6 Key Findings

- x86 provides superior CPU virtualization efficiency due to mature hardware support
- ARM demonstrates competitive disk performance under optimized configurations
- Network virtualization remains a bottleneck across both architectures
- Virtualization overhead is highly sensitive to workload type and system configuration

6.7 Conclusion

The comparative analysis shows that no single architecture uniformly outperforms the other across all subsystems.

x86 excels in compute performance due to advanced hardware virtualization, while ARM can achieve competitive I/O performance when using efficient storage configurations.

However, both architectures suffer from significant network overhead under NAT-based virtualization, highlighting a fundamental limitation of current virtualization networking approaches.

These results emphasize that virtualization performance is determined not only by hardware architecture but also by hypervisor design, storage format, and networking configuration.

Conclusion and Future Work

7.1 Conclusion

This study investigated virtualization overhead across ARM and x86 architectures by evaluating CPU, disk I/O, and network performance under controlled benchmarking conditions.

The results demonstrate that virtualization overhead is highly subsystem-dependent rather than uniform across the system.

7.1.1 Key Findings

- **CPU Performance:** Virtualization overhead is minimal on x86 (1–4%) due to mature hardware-assisted virtualization (VT-x/AMD-V). In contrast, ARM shows higher overhead (∼12%) and increased variability, primarily due to additional software layers and architectural differences.
- **Disk I/O Performance:** Disk performance is significantly affected by virtualization on both platforms, particularly for random and write-heavy workloads. On x86, overhead is severe due to the VDI storage format and double filesystem traversal, while ARM achieves better results using raw disk images.
- **Network Performance:** Network virtualization introduces the largest relative overhead on both architectures. Throughput is reduced by approximately 50%, latency increases by 7–8×, and parallel scalability is limited due to NAT-based networking.
- **Workload Sensitivity:** Virtualization overhead increases for:
  - Small, random operations (disk and network)
  - Synchronous operations (e.g., fsync)
  - High-concurrency workloads (parallel network streams)

7.1.2 Overall Insight

The study highlights that modern virtualization technologies are highly optimized for compute workloads but remain constrained for I/O-intensive applications.

While x86 demonstrates near-native CPU performance, both architectures exhibit significant overhead in storage and networking due to abstraction layers and virtualization design choices.

Thus, virtualization is best suited for compute-bound workloads, while performance-critical I/O workloads require careful configuration and optimization.

7.2 Future Work

This work can be extended in several directions to deepen the understanding of virtualization performance:

7.2.1 Alternative Virtualization Configurations

- Evaluate bridged networking instead of NAT to reduce network overhead
- Compare raw disk images with dynamically allocated formats (VDI, VMDK)
- Investigate paravirtualized drivers and hardware passthrough techniques

7.2.2 Advanced Virtualization Technologies

- Study container-based virtualization (Docker) as a lightweight alternative
- Evaluate KVM on Linux for x86 to compare with VirtualBox
- Explore ARM-native hypervisors beyond QEMU/UTM

7.2.3 Microarchitectural Analysis

- Use hardware performance counters (e.g., `perf`) to analyze:
  - Instructions per cycle (IPC)
  - Cache miss rates
  - Branch prediction behaviour
- Quantify VM-exit frequency and its impact on performance

7.2.4 Real-World Workloads

- Evaluate database workloads (e.g., PostgreSQL, MySQL)
- Benchmark web server performance under virtualization
- Study distributed systems and microservices performance

7.2.5 Scalability Studies

- Analyze performance under higher vCPU counts
- Study multi-VM contention on shared hardware
- Evaluate NUMA-aware scheduling effects

7.2.6 Energy Efficiency

- Compare power consumption between bare metal and VM
- Evaluate energy-performance trade-offs across architectures

7.3 Final Remarks

Virtualization continues to be a fundamental technology in modern computing, enabling cloud infrastructure, development environments, and system isolation.

This study demonstrates that while virtualization has reached near-native efficiency for CPU workloads, I/O subsystems remain a critical bottleneck.

Future improvements in hypervisor design, storage abstraction, and networking models will be essential to achieving truly uniform virtualization performance across all system components.

References